

# CNN

PPT Slides from Deep Learning Specialization (Coursera)

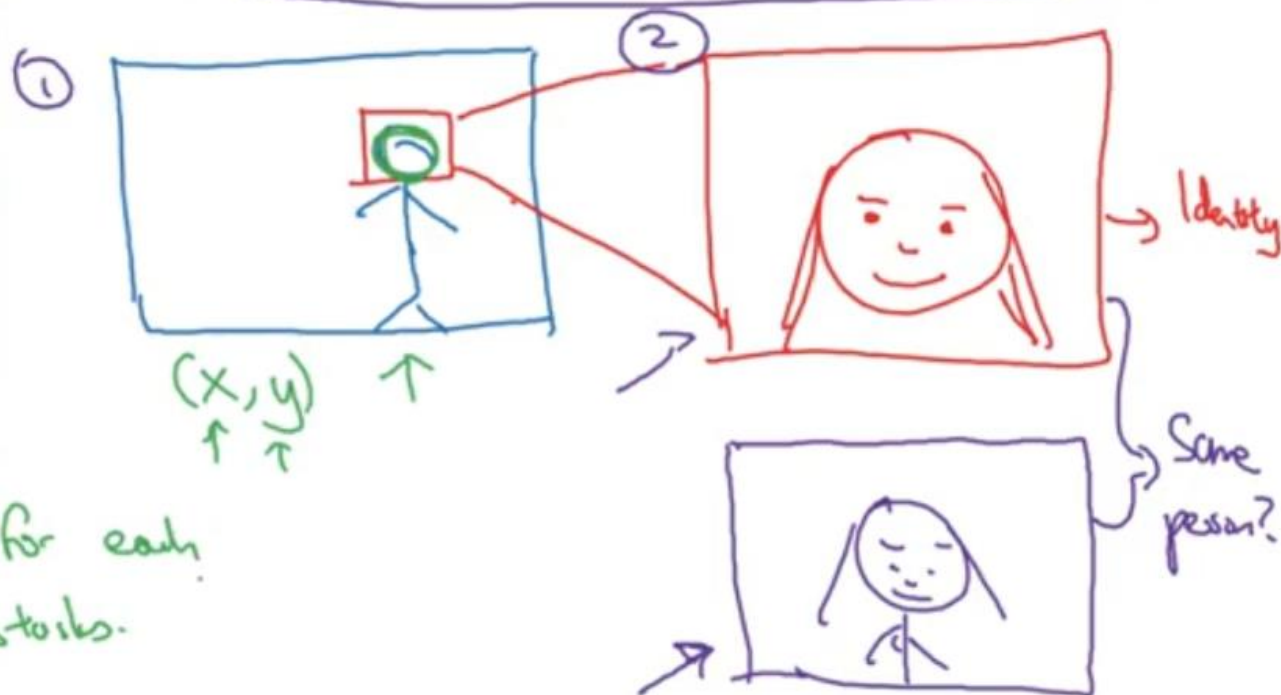
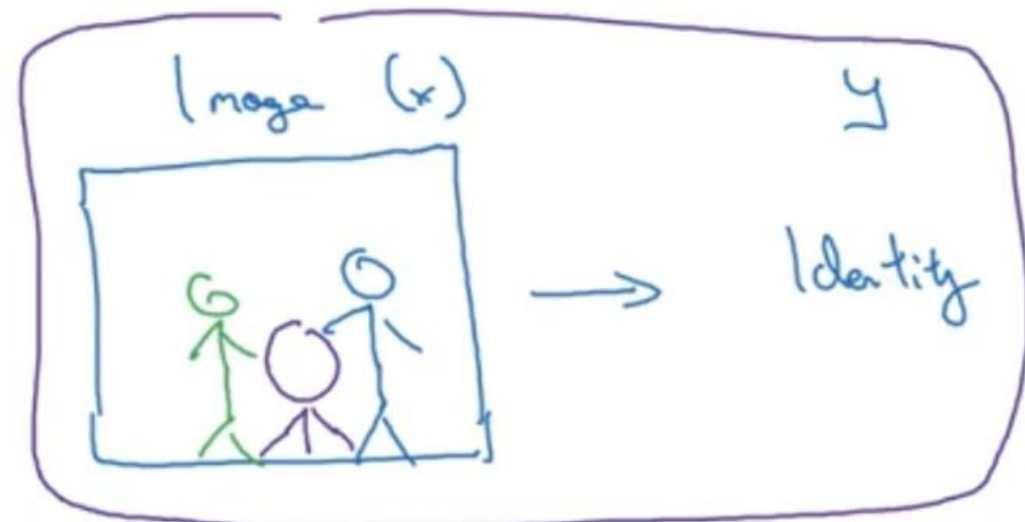
By

Prof. Andrew Ng.

# Face recognition



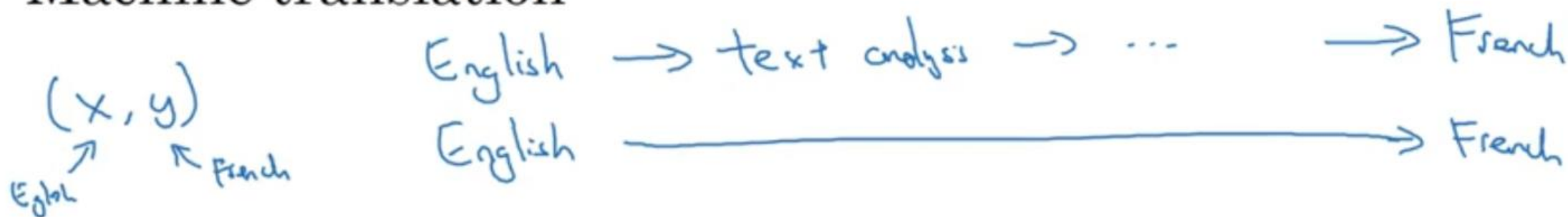
[Image courtesy of Baidu]



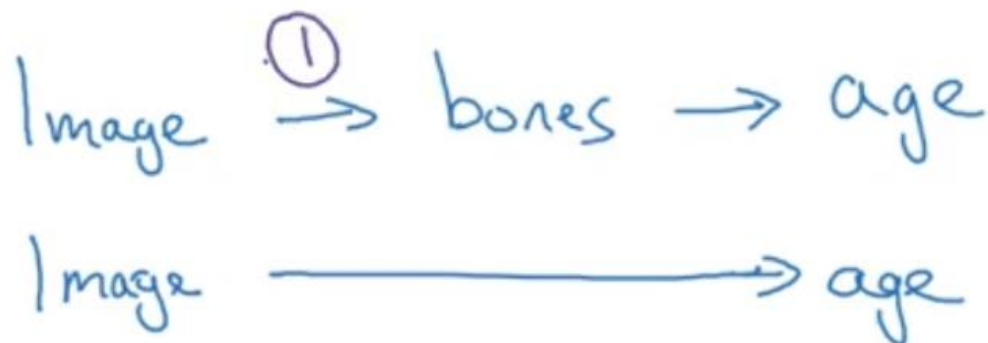
Have data for each  
of 2 sub-tasks.

# More examples

## Machine translation



## Estimating child's age:



# Pros and cons of end-to-end deep learning

Pros:

- Let the data speak
- Less hand-designing of components needed

$x \rightarrow y$  → "phonemes"  
cat

Cons:

- May need large amount of data
- Excludes potentially useful hand-designed components

$x - - - - - \rightarrow y$  input end  
↓  
 $x \rightarrow y$  output end  
↓  
(x, y)

Data. Hand-design.

Andrew Ng

# Applying end-to-end deep learning

Key question: Do you have sufficient data to learn a function of the complexity needed to map  $x$  to  $y$ ?

$x \rightarrow y$

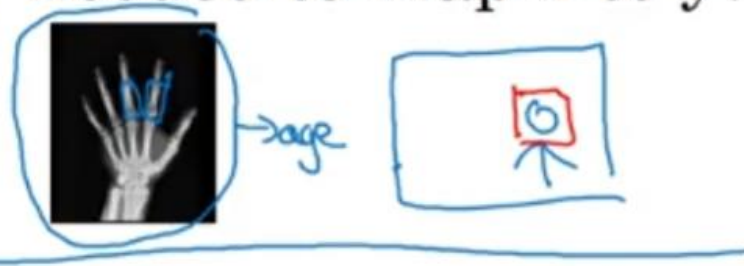


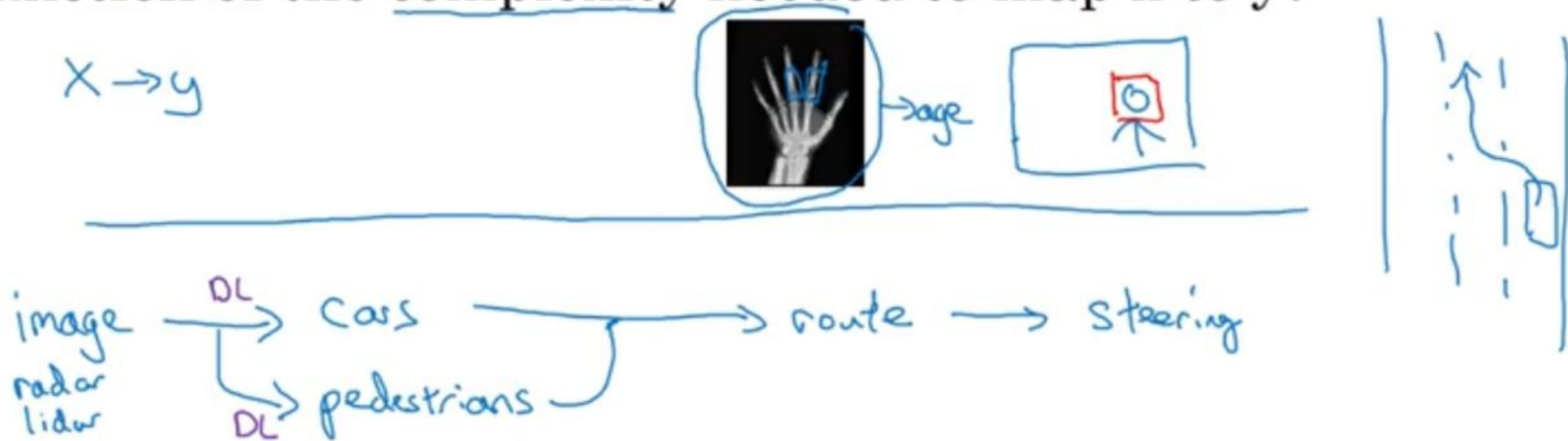
image  
radar  
lidar

Andrew Ng

other sensor readings as well,

# Applying end-to-end deep learning

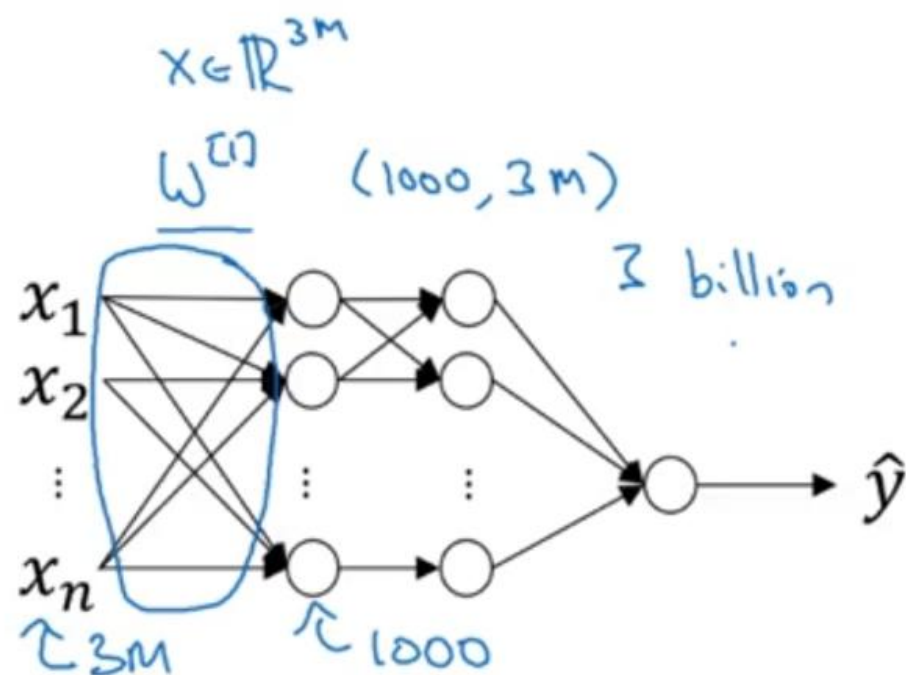
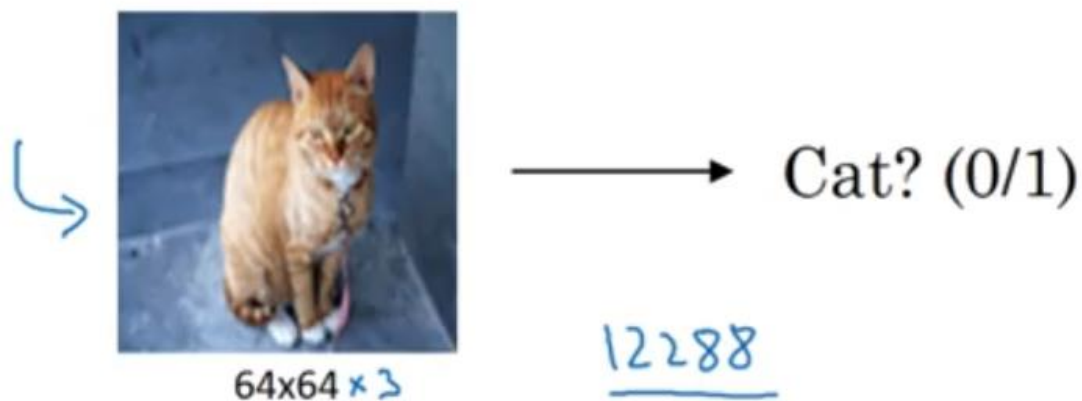
Key question: Do you have sufficient data to learn a function of the complexity needed to map  $x$  to  $y$ ?



Andrew Ng

And if you ever take a course in robotics you'll learn about motion planning.

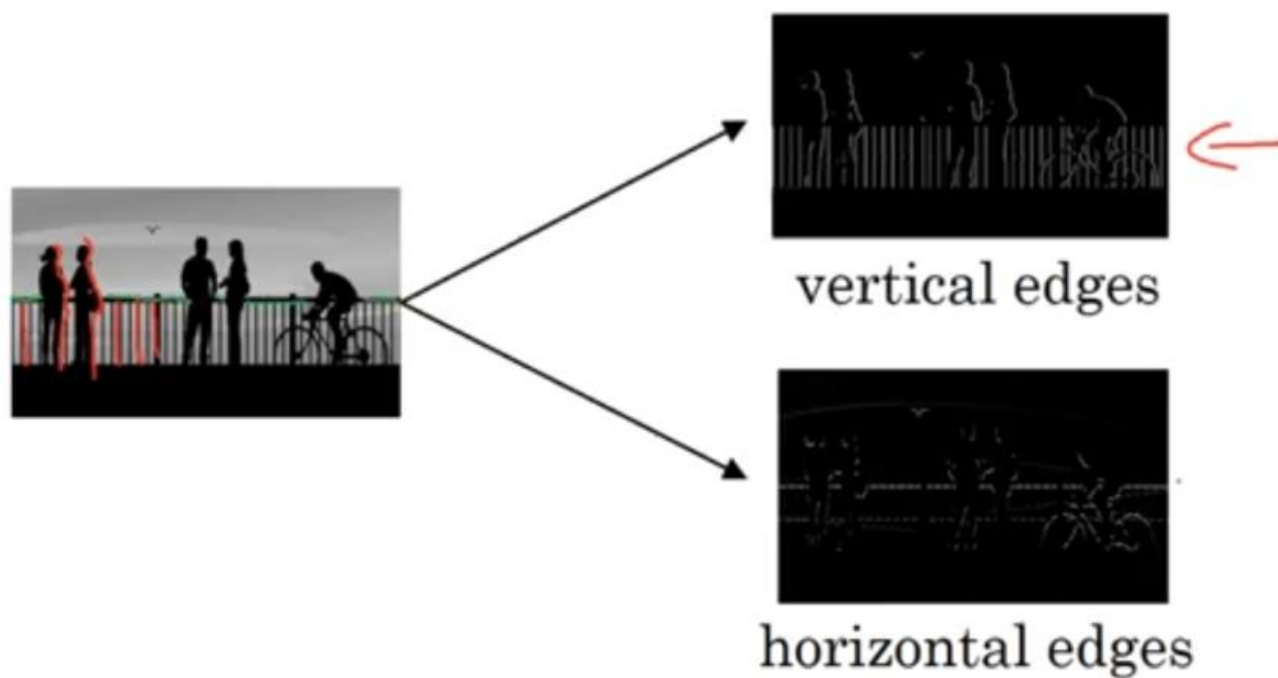
# Deep Learning on large images



Andrew Ng

And also, the computational requirements and the memory requirements to train

# Computer Vision Problem



Andrew Ng

this railing is and that also gets detected sort of roughly here.

# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 7 | 4 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

6x6

"convolution"  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|    |  |  |  |
|----|--|--|--|
| -5 |  |  |  |
|    |  |  |  |
|    |  |  |  |
|    |  |  |  |

4x4

# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|   |                       |                       |                        |   |   |
|---|-----------------------|-----------------------|------------------------|---|---|
| 3 | <u>0</u> <sup>1</sup> | <u>1</u> <sup>0</sup> | <u>2</u> <sup>-1</sup> | 7 | 4 |
| 1 | <u>5</u> <sup>1</sup> | <u>8</u> <sup>0</sup> | <u>9</u> <sup>-1</sup> | 3 | 1 |
| 2 | <u>7</u> <sup>1</sup> | <u>2</u> <sup>0</sup> | <u>5</u> <sup>-1</sup> | 1 | 3 |
| 0 | 1                     | 3                     | 1                      | 7 | 8 |
| 4 | 2                     | 1                     | 6                      | 2 | 8 |
| 2 | 4                     | 5                     | 2                      | 3 | 9 |

6x6

"convolution"  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|           |           |  |  |
|-----------|-----------|--|--|
| <u>-5</u> | <u>-4</u> |  |  |
|           |           |  |  |
|           |           |  |  |
|           |           |  |  |

4x4

# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|   |          |          |                       |                |                 |
|---|----------|----------|-----------------------|----------------|-----------------|
| 3 | <u>0</u> | <u>1</u> | <u>2</u> <sup>1</sup> | 7 <sup>0</sup> | 4 <sup>-1</sup> |
| 1 | <u>5</u> | <u>8</u> | <u>9</u> <sup>1</sup> | 3 <sup>0</sup> | 1 <sup>-1</sup> |
| 2 | <u>7</u> | <u>2</u> | <u>5</u> <sup>1</sup> | 1 <sup>0</sup> | 3 <sup>-1</sup> |
| 0 | 1        | 3        | 1                     | 7              | 8               |
| 4 | 2        | 1        | 6                     | 2              | 8               |
| 2 | 4        | 5        | 2                     | 3              | 9               |

6x6

"convolution"  
↓  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|           |           |   |   |
|-----------|-----------|---|---|
| <u>-5</u> | <u>-4</u> | 0 | 8 |
|           |           |   |   |
|           |           |   |   |
|           |           |   |   |

4x4

# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|                       |                       |                        |          |          |          |
|-----------------------|-----------------------|------------------------|----------|----------|----------|
| 3                     | <u>0</u>              | <u>1</u>               | <u>2</u> | <u>7</u> | <u>4</u> |
| <sup>1</sup> <u>1</u> | <sup>0</sup> <u>5</u> | <sup>-1</sup> <u>8</u> | <u>9</u> | <u>3</u> | <u>1</u> |
| <sup>1</sup> <u>2</u> | <sup>0</sup> <u>7</u> | <sup>-1</sup> <u>2</u> | <u>5</u> | <u>1</u> | <u>3</u> |
| <sup>1</sup> <u>0</u> | <sup>0</sup> <u>1</u> | <sup>-1</sup> <u>3</u> | 1        | 7        | 8        |
| 4                     | 2                     | 1                      | 6        | 2        | 8        |
| 2                     | 4                     | 5                      | 2        | 3        | 9        |

6x6

"convolution"  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|           |           |   |          |
|-----------|-----------|---|----------|
| <u>-5</u> | <u>-4</u> | 0 | <u>8</u> |
|           |           |   |          |
|           |           |   |          |
|           |           |   |          |

4x4

# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|                |                       |                        |          |          |          |
|----------------|-----------------------|------------------------|----------|----------|----------|
| 3              | <u>0</u>              | <u>1</u>               | <u>2</u> | <u>7</u> | <u>4</u> |
| <sup>1</sup> 1 | <sup>0</sup> <u>5</u> | <sup>-1</sup> <u>8</u> | <u>9</u> | <u>3</u> | <u>1</u> |
| <sup>1</sup> 2 | <sup>0</sup> <u>7</u> | <sup>-1</sup> <u>2</u> | <u>5</u> | <u>1</u> | <u>3</u> |
| <sup>1</sup> 0 | <sup>0</sup> <u>1</u> | <sup>-1</sup> <u>3</u> | 1        | 7        | 8        |
| 4              | 2                     | 1                      | 6        | 2        | 8        |
| 2              | 4                     | 5                      | 2        | 3        | 9        |

6x6

"convolution"  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|            |           |   |          |
|------------|-----------|---|----------|
| <u>-5</u>  | <u>-4</u> | 0 | <u>8</u> |
| <u>-10</u> |           |   |          |
|            |           |   |          |
|            |           |   |          |

4x4



# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|   |          |          |          |          |          |
|---|----------|----------|----------|----------|----------|
| 3 | <u>0</u> | <u>1</u> | <u>2</u> | <u>7</u> | <u>4</u> |
| 1 | <u>5</u> | <u>8</u> | <u>9</u> | <u>3</u> | <u>1</u> |
| 2 | <u>7</u> | <u>2</u> | <u>5</u> | <u>1</u> | <u>3</u> |
| 0 | 1        | 3        | 1        | 7        | 8        |
| 4 | 2        | 1        | 6        | 2        | 8        |
| 2 | 4        | 5        | 2        | 3        | 9        |

6x6

"convolution"  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|            |           |   |          |
|------------|-----------|---|----------|
| <u>-5</u>  | <u>-4</u> | 0 | <u>8</u> |
| <u>-10</u> | -2        | . |          |
|            |           |   |          |
|            |           |   |          |

4x4

# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|   |          |          |                       |                       |                        |
|---|----------|----------|-----------------------|-----------------------|------------------------|
| 3 | <u>0</u> | <u>1</u> | <u>2</u>              | <u>7</u>              | <u>4</u>               |
| 1 | <u>5</u> | <u>8</u> | <u>9</u> <sup>1</sup> | <u>3</u> <sup>0</sup> | <u>1</u> <sup>-1</sup> |
| 2 | <u>7</u> | <u>2</u> | <u>5</u> <sup>1</sup> | <u>1</u> <sup>0</sup> | <u>3</u> <sup>-1</sup> |
| 0 | 1        | 3        | <u>1</u> <sup>1</sup> | <u>7</u> <sup>0</sup> | <u>8</u> <sup>-1</sup> |
| 4 | 2        | 1        | 6                     | 2                     | 8                      |
| 2 | 4        | 5        | 2                     | 3                     | 9                      |

6x6

"convolution"

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|            |           |   |          |
|------------|-----------|---|----------|
| <u>-5</u>  | <u>-4</u> | 0 | <u>8</u> |
| <u>-10</u> | -2        | 2 |          |
|            |           |   |          |
|            |           |   |          |

4x4



# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|   |          |          |          |          |          |
|---|----------|----------|----------|----------|----------|
| 3 | <u>0</u> | <u>1</u> | <u>2</u> | <u>7</u> | <u>4</u> |
| 1 | <u>5</u> | <u>8</u> | <u>9</u> | <u>3</u> | <u>1</u> |
| 2 | <u>7</u> | <u>2</u> | <u>5</u> | <u>1</u> | <u>3</u> |
| 0 | 1        | 3        | 1        | 7        | 8        |
| 4 | 2        | 1        | 6        | 2        | 8        |
| 2 | 4        | 5        | 2        | 3        | 9        |

6x6

"convolution"  
↓  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|            |           |    |          |
|------------|-----------|----|----------|
| <u>-5</u>  | <u>-4</u> | 0  | <u>8</u> |
| <u>-10</u> | -2        | 2  | 3        |
| 0          | -2        | -4 | -7       |
| -3         | -2        | -3 | -16      |

4x4

# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|   |          |          |          |          |          |
|---|----------|----------|----------|----------|----------|
| 3 | <u>0</u> | <u>1</u> | <u>2</u> | <u>7</u> | <u>4</u> |
| 1 | <u>5</u> | <u>8</u> | <u>9</u> | <u>3</u> | <u>1</u> |
| 2 | <u>7</u> | <u>2</u> | <u>5</u> | <u>1</u> | <u>3</u> |
| 0 | 1        | 3        | 1        | 7        | 8        |
| 4 | 2        | 1        | 6        | 2        | 8        |
| 2 | 4        | 5        | 2        | 3        | 9        |

6x6

"convolution"  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|            |           |    |            |
|------------|-----------|----|------------|
| <u>-5</u>  | <u>-4</u> | 0  | <u>8</u>   |
| <u>-10</u> | -2        | 2  | 3          |
| 0          | -2        | -4 | -7         |
| -3         | -2        | -3 | <u>-16</u> |

4x4

# Vertical edge detection

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 3 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

|   |          |          |          |          |          |
|---|----------|----------|----------|----------|----------|
| 3 | <u>0</u> | <u>1</u> | <u>2</u> | <u>7</u> | <u>4</u> |
| 1 | <u>5</u> | <u>8</u> | <u>9</u> | <u>3</u> | <u>1</u> |
| 2 | <u>7</u> | <u>2</u> | <u>5</u> | <u>1</u> | <u>3</u> |
| 0 | 1        | 3        | 1        | 7        | 8        |
| 4 | 2        | 1        | 6        | 2        | 8        |
| 2 | 4        | 5        | 2        | 3        | 9        |

6x6

"convolution"  
↓  
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3  
filter

=

|            |           |    |            |
|------------|-----------|----|------------|
| <u>-5</u>  | <u>-4</u> | 0  | <u>8</u>   |
| <u>-10</u> | -2        | 2  | 3          |
| 0          | -2        | -4 | -7         |
| -3         | -2        | -3 | <u>-16</u> |

4x4

python: conv-forward  
tensorflow: tf.nn.conv2d  
keras: Conv2D

# Vertical edge detection

↓

|    |    |    |   |   |   |
|----|----|----|---|---|---|
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |

6x6

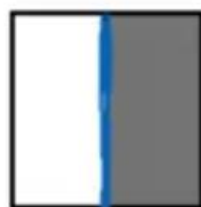
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

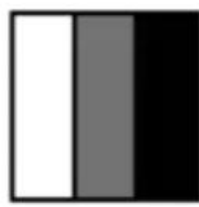
3x3

=

|   |    |    |   |
|---|----|----|---|
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |



\*



↑ ↑ ↑

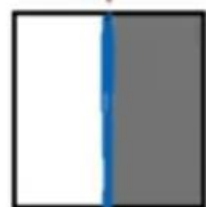
Andrew Ng

the left column 10 plus 10 plus 10 and then zeroes in the middle and then -10,

# Vertical edge detection

|    |           |           |          |   |   |
|----|-----------|-----------|----------|---|---|
| 10 | 10        | 10        | 0        | 0 | 0 |
| 10 | 10        | 10        | 0        | 0 | 0 |
| 10 | 10        | 10        | 0        | 0 | 0 |
| 10 | <u>10</u> | <u>10</u> | <u>0</u> | 0 | 0 |
| 10 | <u>10</u> | <u>10</u> | <u>0</u> | 0 | 0 |
| 10 | <u>10</u> | <u>10</u> | <u>0</u> | 0 | 0 |

6x6



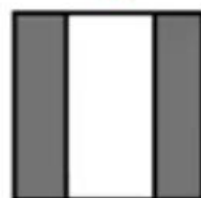
\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

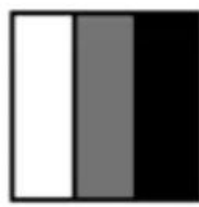
3x3

=

|   |           |    |   |
|---|-----------|----|---|
| 0 | 30        | 30 | 0 |
| 0 | 30        | 30 | 0 |
| 0 | 30        | 30 | 0 |
| 0 | <u>30</u> | 30 | 0 |



\*



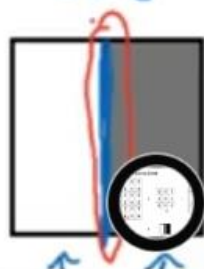
Andrew Ng

the middle and that corresponds to this having

# Vertical edge detection

|    |    |    |   |   |   |
|----|----|----|---|---|---|
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |

6x6



\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

3x3

=

|   |    |    |   |
|---|----|----|---|
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |



\*



# Vertical edge detection examples

|    |    |    |   |   |   |
|----|----|----|---|---|---|
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |



|   |   |   |    |    |    |
|---|---|---|----|----|----|
| 0 | 0 | 0 | 10 | 10 | 10 |
| 0 | 0 | 0 | 10 | 10 | 10 |
| 0 | 0 | 0 | 10 | 10 | 10 |
| 0 | 0 | 0 | 10 | 10 | 10 |
| 0 | 0 | 0 | 10 | 10 | 10 |
| 0 | 0 | 0 | 10 | 10 | 10 |



\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |



=

|   |    |    |   |
|---|----|----|---|
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |



\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |



=

|   |     |     |   |
|---|-----|-----|---|
| 0 | -30 | -30 | 0 |
| 0 | -30 | -30 | 0 |
| 0 | -30 | -30 | 0 |
| 0 | -30 | -30 | 0 |



you can plot that as a picture  
that maybe looks like that.

# Vertical and Horizontal Edge Detection

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

Vertical

|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

Horizontal

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 10 | 10 | 10 | 0  | 0  | 0  |
| 10 | 10 | 10 | 0  | 0  | 0  |
| 10 | 10 | 10 | 0  | 0  | 0  |
| 0  | 0  | 0  | 10 | 10 | 10 |
| 0  | 0  | 0  | 10 | 10 | 10 |
| 0  | 0  | 0  | 10 | 10 | 10 |

\*

|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

=

|    |    |     |     |
|----|----|-----|-----|
| 0  | 0  | 0   | 0   |
| 30 | 10 | -10 | -30 |
| 30 | 10 | -10 | -30 |
| 0  | 0  | 0   | 0   |



# Vertical and Horizontal Edge Detection

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

Vertical

|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

Horizontal

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 10 | 10 | 10 | 0  | 0  | 0  |
| 10 | 10 | 10 | 0  | 0  | 0  |
| 10 | 10 | 10 | 0  | 0  | 0  |
| 0  | 0  | 0  | 10 | 10 | 10 |
| 0  | 0  | 0  | 10 | 10 | 10 |
| 0  | 0  | 0  | 10 | 10 | 10 |

\*

|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

=

|    |    |     |     |
|----|----|-----|-----|
| 0  | 0  | 0   | 0   |
| 30 | 10 | -10 | -30 |
| 30 | 10 | -10 | -30 |
| 0  | 0  | 0   | 0   |



# Vertical and Horizontal Edge Detection

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

Vertical

|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

Horizontal

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 10 | 10 | 10 | 0  | 0  | 0  |
| 10 | 10 | 10 | 0  | 0  | 0  |
| 10 | 10 | 10 | 0  | 0  | 0  |
| 0  | 0  | 0  | 10 | 10 | 10 |
| 0  | 0  | 0  | 10 | 10 | 10 |
| 0  | 0  | 0  | 10 | 10 | 10 |

6x6

\*

|    |    |    |
|----|----|----|
| 1  | 1  | 1  |
| 0  | 0  | 0  |
| -1 | -1 | -1 |

=

|    |    |     |     |
|----|----|-----|-----|
| 0  | 0  | 0   | 0   |
| 30 | 10 | -10 | -30 |
| 30 | 10 | -10 | -30 |
| 0  | 0  | 0   | 0   |



say a thousand by a thousand image with  
this type of checkerboard pattern,

# Learning to detect edges

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

→

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |

Sobel filter

|    |   |     |
|----|---|-----|
| 3  | 0 | -3  |
| 10 | 0 | -10 |
| 3  | 0 | -3  |

Scharr filter

|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 7 | 4 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

\*

|       |       |       |
|-------|-------|-------|
| $w_1$ | $w_2$ | $w_3$ |
| $w_4$ | $w_5$ | $w_6$ |
| $w_7$ | $w_8$ | $w_9$ |

=

|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

parameters, the backprop can  
choose to learn 1, 1, 1, 0, 0, 0,

# Learning to detect edges

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 7 | 4 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

→

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |

Sobel filter



|       |       |       |
|-------|-------|-------|
| $w_1$ | $w_2$ | $w_3$ |
| $w_4$ | $w_5$ | $w_6$ |
| $w_7$ | $w_8$ | $w_9$ |



|    |   |     |
|----|---|-----|
| 3  | 0 | -3  |
| 10 | 0 | -10 |
| 3  | 0 | -3  |

Scharr filter



|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

And rather than just vertical and horizontal edges,

# Learning to detect edges

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 7 | 4 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

→

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |

Sobel filter



|       |       |       |
|-------|-------|-------|
| $w_1$ | $w_2$ | $w_3$ |
| $w_4$ | $w_5$ | $w_6$ |
| $w_7$ | $w_8$ | $w_9$ |



=

45°  
70°  
73°

|    |   |     |
|----|---|-----|
| 3  | 0 | -3  |
| 10 | 0 | -10 |
| 3  | 0 | -3  |

Scharr filter



|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

And so by just letting all of these numbers be parameters and learning them

# Learning to detect edges

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 3 | 0 | 1 | 2 | 7 | 4 |
| 1 | 5 | 8 | 9 | 3 | 1 |
| 2 | 7 | 2 | 5 | 1 | 3 |
| 0 | 1 | 3 | 1 | 7 | 8 |
| 4 | 2 | 1 | 6 | 2 | 8 |
| 2 | 4 | 5 | 2 | 3 | 9 |

→

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |

Sobel filter



|       |       |       |
|-------|-------|-------|
| $w_1$ | $w_2$ | $w_3$ |
| $w_4$ | $w_5$ | $w_6$ |
| $w_7$ | $w_8$ | $w_9$ |



=

45°  
70°  
73°

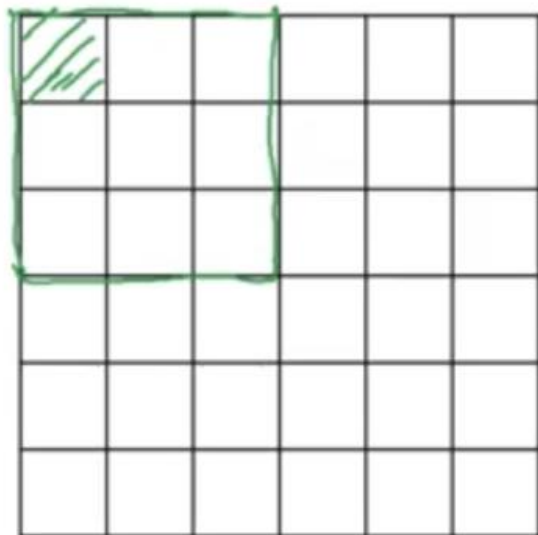
|    |   |     |
|----|---|-----|
| 3  | 0 | -3  |
| 10 | 0 | -10 |
| 3  | 0 | -3  |

Scharr filter

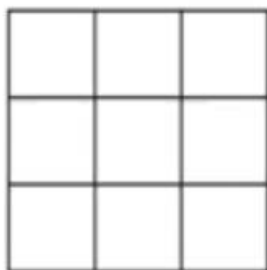


|  |  |  |  |
|--|--|--|--|
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |
|  |  |  |  |

# Padding



\*



=

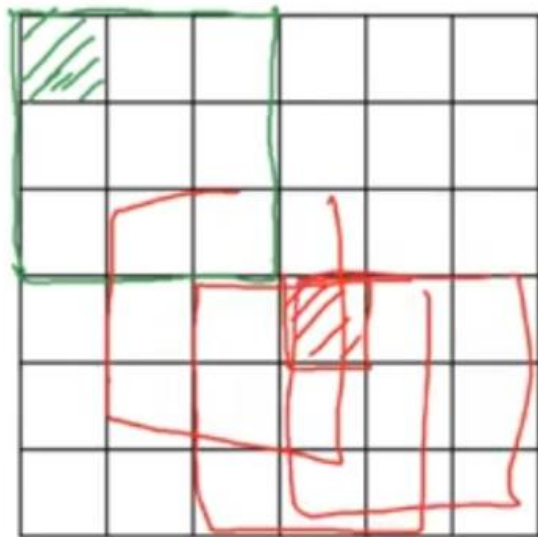
$3 \times 3$   
 $f \times f$

$\frac{6 \times 6}{n \times n}$

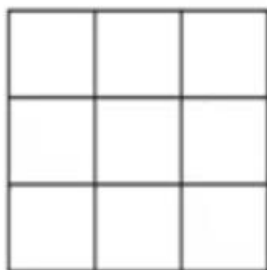
$n - f + 1 \times n - f + 1$   
 $6 - 3 + 1 = 4$

$\rightarrow \underline{\underline{4 \times 4}}$

# Padding



\*



=

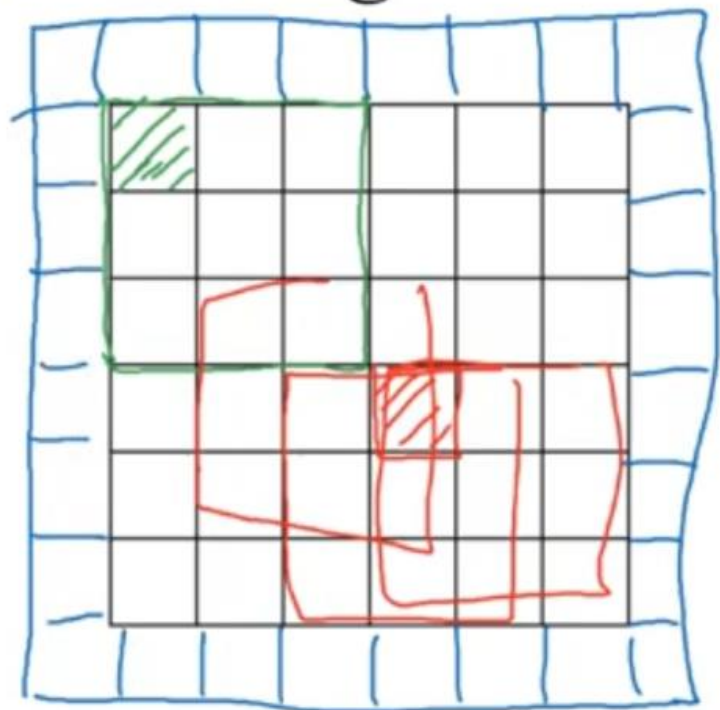
$3 \times 3$   
 $f \times f$

$\frac{6 \times 6}{n \times n}$

$n - f + 1 \times n - f + 1$   
 $6 - 3 + 1 = 4$

$\rightarrow \underline{\underline{4 \times 4}}$

# Padding



- shrinky output
- throw away info from edge

$$\begin{array}{c} * \end{array} \begin{array}{|c|c|c|} \hline & & \\ \hline & & \\ \hline & & \\ \hline \end{array} =$$

$3 \times 3$   
 $f \times f$

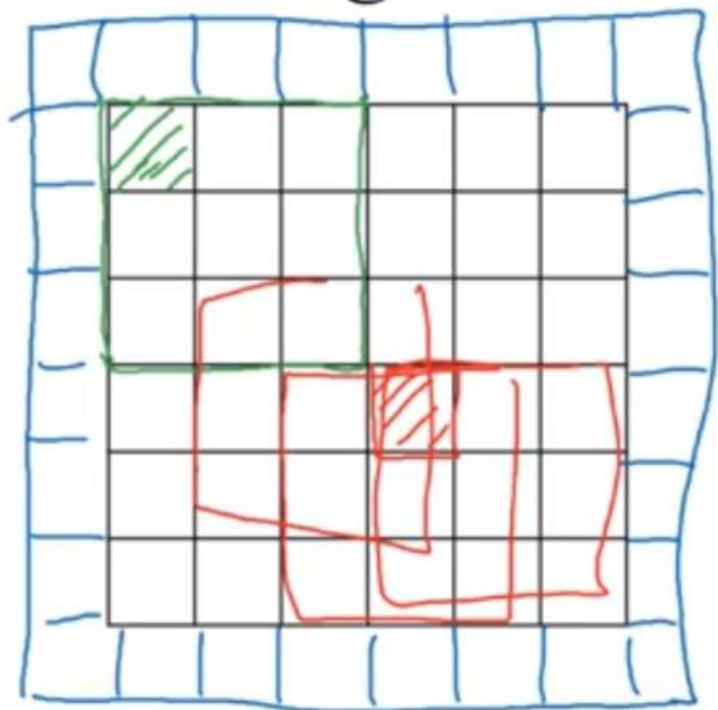
$$\frac{6 \times 6}{n \times n} \rightarrow 8$$

$$n - f + 1 \times n - f + 1$$
$$6 - 3 + 1 = 4$$

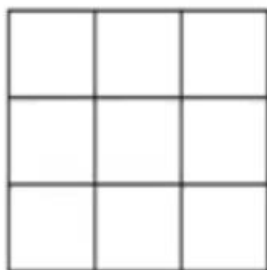
$$\rightarrow \underline{\underline{4 \times 4}}$$

# Padding

- shrinky output
- throw away info from edge

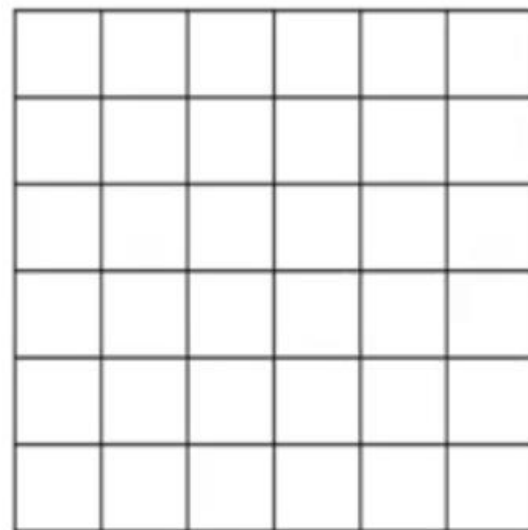


\*



3x3  
f x f

=



6x6

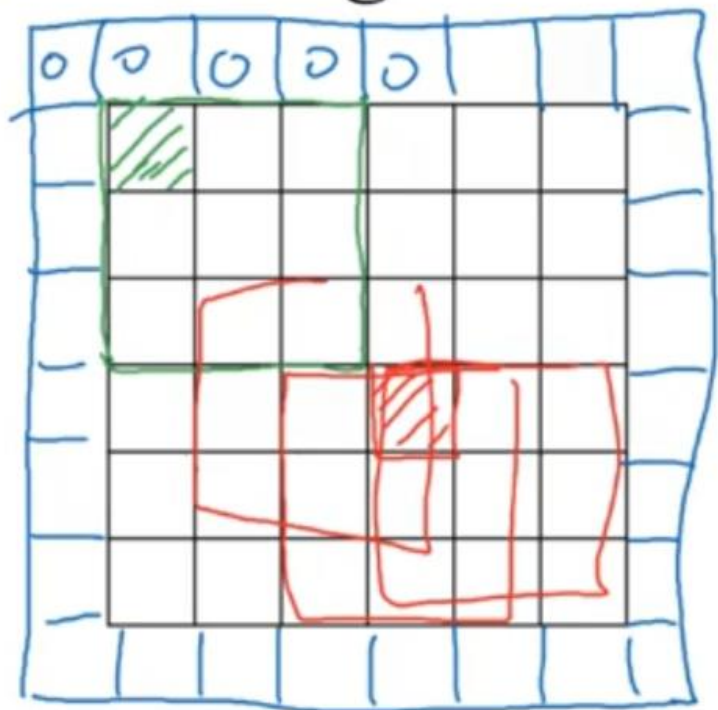
6x6 → 8x8  
n x n

$n - f + 1 \times n - f + 1$   
 $6 - 3 + 1 = 4$

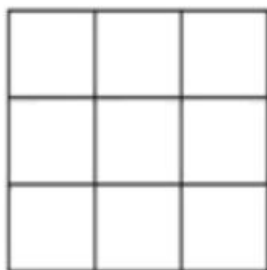
→ ~~4x4~~

# Padding

- shrinky output
- throw away info from edge

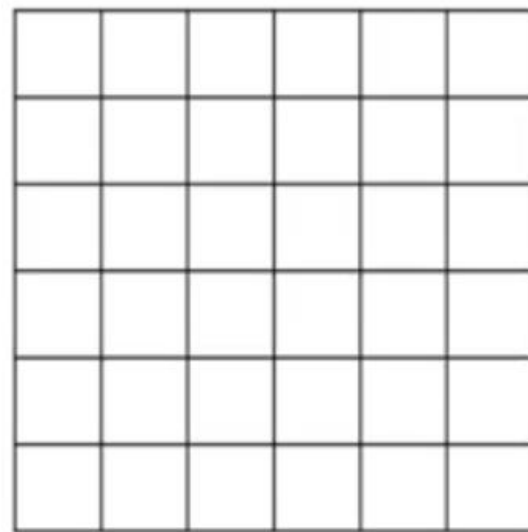


\*



3x3  
f x f

=



6x6

$$\frac{6 \times 6}{n \times n} \rightarrow 8 \times 8$$

$$n - f + 1 \times n - f + 1$$
$$6 - 3 + 1 = 4$$

→ ~~4x4~~

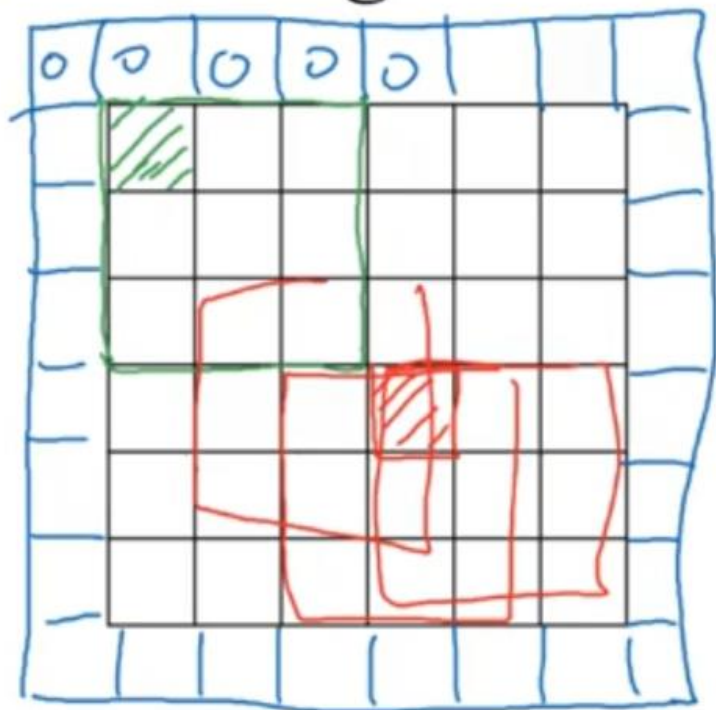
$$p = \text{padding} = 1$$

Andrew Ng

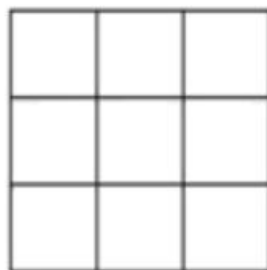
because we're padding all around with an extra boarder of one pixels,

# Padding

- shrinky output
- throw away info from edge

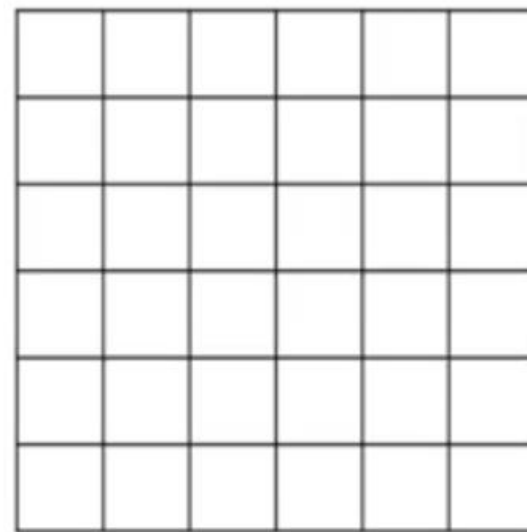


\*



3x3  
f x f

=



6x6

$$\frac{6 \times 6}{n \times n} \rightarrow 8 \times 8$$

$$p = \text{padding} = 1$$

$$n - f + 1 \times n - f + 1$$

$$6 - 3 + 1 = 4$$

$$n + 2p - f + 1 \times n + 2p - f + 1$$

$$6 + 2 - 3 + 1 = 6$$

→ ~~4x4~~

Andrew Ng

So, this becomes six plus two times one minus three plus one by the same thing on that.

# Valid and Same convolutions

→ no padding

$$\begin{array}{lclclcl} \text{"Valid":} & n \times n & * & f \times f & \rightarrow & \frac{n-f+1}{1} \times \frac{n-f+1}{1} \\ & 6 \times 6 & * & 3 \times 3 & \rightarrow & 4 \times 4 \end{array}$$

"Same": Pad so that output size is the same as the input size.

# Valid and Same convolutions

→ no padding

“Valid”:  $n \times n$   $\times$   $f \times f$   $\rightarrow \underline{n - f + 1} \times n - f + 1$   
 $6 \times 6$   $\times$   $3 \times 3$   $\rightarrow$   $4 \times 4$

“Same”: Pad so that output size is the same as the input size.

$$n + 2p - f + 1 \times n + 2p - f + 1$$
$$\cancel{n + 2p - f + 1} = \cancel{n} \Rightarrow p = \frac{f - 1}{2}$$

# Valid and Same convolutions

→ no padding

“Valid”:  $n \times n$   $\ast$   $f \times f$   $\rightarrow \underline{n - f + 1} \times n - f + 1$   
 $6 \times 6$   $\ast$   $3 \times 3$   $\rightarrow$   $4 \times 4$

“Same”: Pad so that output size is the same as the input size.

$$\begin{aligned} n + 2p - f + 1 &\times n + 2p - f + 1 \\ \cancel{n + 2p - f + 1} &= \cancel{n} \Rightarrow \underline{p} = \frac{f-1}{2} \\ 3 \times 3 \quad p &= \frac{3-1}{2} = 1 \end{aligned}$$

# Valid and Same convolutions

→ no padding

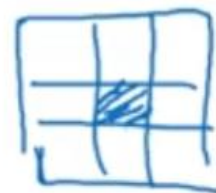
“Valid”:  $n \times n$   $\ast$   $f \times f$   $\rightarrow \frac{n-f+1}{1} \times n-f+1$   
 $6 \times 6$   $\ast$   $3 \times 3$   $\rightarrow 4 \times 4$

“Same”: Pad so that output size is the same as the input size.

$$n+2p-f+1 \times n+2p-f+1$$
$$\cancel{n+2p-f+1} = \cancel{n} \Rightarrow \boxed{p = \frac{f-1}{2}}$$

$$3 \times 3 \quad p = \frac{3-1}{2} = 1 \quad \left| \quad \begin{matrix} 5 \times 5 \\ f=5 \end{matrix} \quad p=2$$

$f$  is usually odd



Andrew Ng

# Strided convolution

|   |   |                 |                |                |   |   |
|---|---|-----------------|----------------|----------------|---|---|
| 2 | 3 | 7 <sup>3</sup>  | 4 <sup>4</sup> | 6 <sup>4</sup> | 2 | 9 |
| 6 | 6 | 9 <sup>1</sup>  | 8 <sup>0</sup> | 7 <sup>2</sup> | 4 | 3 |
| 3 | 4 | 8 <sup>-1</sup> | 3 <sup>0</sup> | 8 <sup>3</sup> | 9 | 7 |
| 7 | 8 | 3               | 6              | 6              | 3 | 4 |
| 4 | 2 | 1               | 8              | 3              | 4 | 6 |
| 3 | 2 | 4               | 1              | 9              | 8 | 3 |
| 0 | 1 | 3               | 9              | 2              | 1 | 4 |

7x7

\*

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

3x3

Stride = 2

=

|    |  |  |
|----|--|--|
| 91 |  |  |
|    |  |  |
|    |  |  |

# Strided convolution

|   |   |                 |                |                |   |   |
|---|---|-----------------|----------------|----------------|---|---|
| 2 | 3 | 7 <sup>3</sup>  | 4 <sup>4</sup> | 6 <sup>4</sup> | 2 | 9 |
| 6 | 6 | 9 <sup>1</sup>  | 8 <sup>0</sup> | 7 <sup>2</sup> | 4 | 3 |
| 3 | 4 | 8 <sup>-1</sup> | 3 <sup>0</sup> | 8 <sup>3</sup> | 9 | 7 |
| 7 | 8 | 3               | 6              | 6              | 3 | 4 |
| 4 | 2 | 1               | 8              | 3              | 4 | 6 |
| 3 | 2 | 4               | 1              | 9              | 8 | 3 |
| 0 | 1 | 3               | 9              | 2              | 1 | 4 |

7x7

\*

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

3x3

Stride = 2

=

|    |  |  |
|----|--|--|
| 91 |  |  |
|    |  |  |
|    |  |  |



# Strided convolution

Diagram illustrating a strided convolution operation. The input is a 7x7 matrix, the kernel is a 3x3 matrix, and the output is a 3x3 matrix. The input matrix is shown with a 3x3 region highlighted in blue, indicating the current receptive field. The kernel is shown as a 3x3 matrix. The output matrix is shown with the first row filled with handwritten values 91, 100, and 83.

|   |   |   |   |                 |                |                |
|---|---|---|---|-----------------|----------------|----------------|
| 2 | 3 | 7 | 4 | 6 <sup>3</sup>  | 2 <sup>4</sup> | 9 <sup>4</sup> |
| 6 | 6 | 9 | 8 | 7 <sup>1</sup>  | 4 <sup>0</sup> | 3 <sup>2</sup> |
| 3 | 4 | 8 | 3 | 8 <sup>-1</sup> | 9 <sup>0</sup> | 7 <sup>3</sup> |
| 7 | 8 | 3 | 6 | 6               | 3              | 4              |
| 4 | 2 | 1 | 8 | 3               | 4              | 6              |
| 3 | 2 | 4 | 1 | 9               | 8              | 3              |
| 0 | 1 | 3 | 9 | 2               | 1              | 4              |

7x7

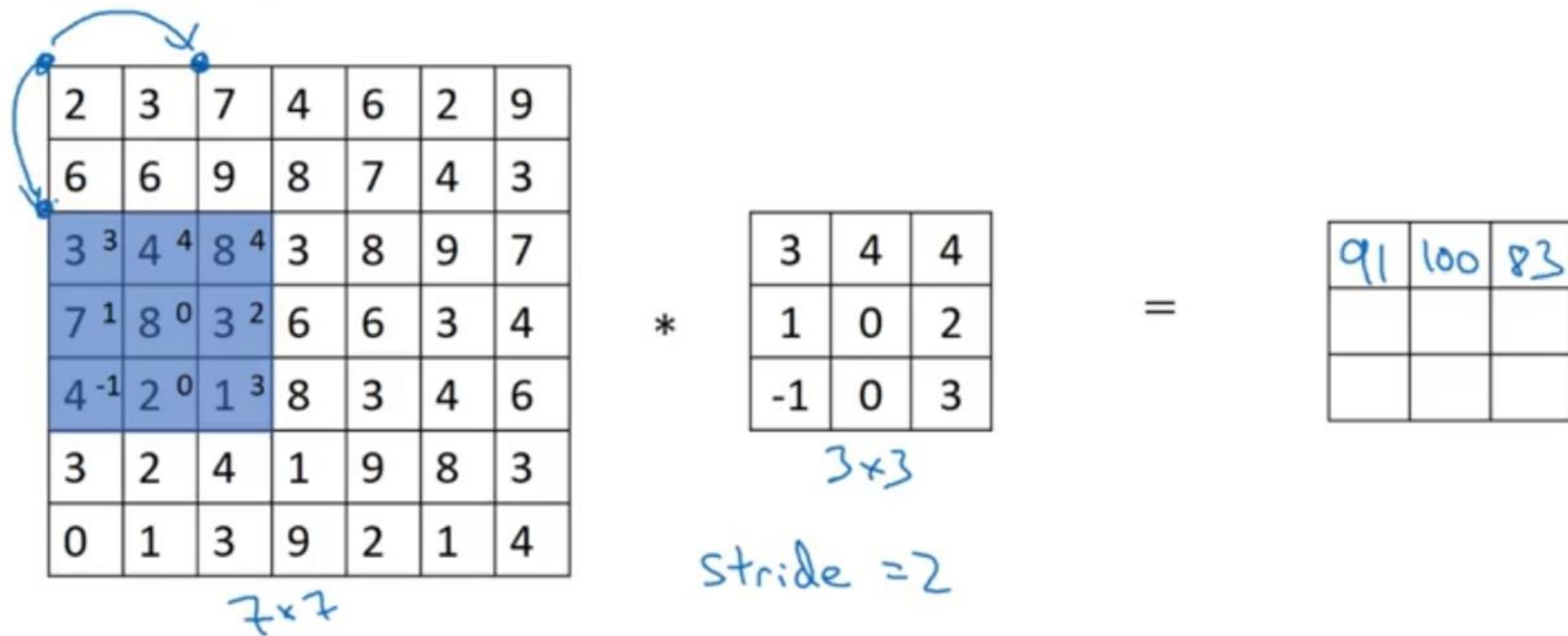
|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

3x3

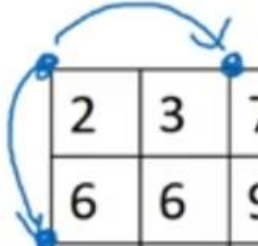
Stride = 2

|    |     |    |
|----|-----|----|
| 91 | 100 | 83 |
|    |     |    |
|    |     |    |

# Strided convolution



# Strided convolution



|   |   |   |   |                 |                |                |
|---|---|---|---|-----------------|----------------|----------------|
| 2 | 3 | 7 | 4 | 6               | 2              | 9              |
| 6 | 6 | 9 | 8 | 7               | 4              | 3              |
| 3 | 4 | 8 | 3 | 8 <sup>3</sup>  | 9 <sup>4</sup> | 7 <sup>4</sup> |
| 7 | 8 | 3 | 6 | 6 <sup>1</sup>  | 3 <sup>0</sup> | 4 <sup>2</sup> |
| 4 | 2 | 1 | 8 | 3 <sup>-1</sup> | 4 <sup>0</sup> | 6 <sup>3</sup> |
| 3 | 2 | 4 | 1 | 9               | 8              | 3              |
| 0 | 1 | 3 | 9 | 2               | 1              | 4              |

7x7

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

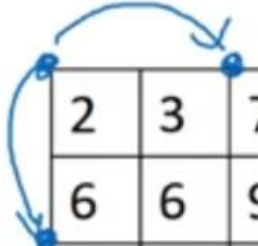
3x3

Stride = 2

=

|    |     |    |
|----|-----|----|
| 91 | 100 | 83 |
| 69 | 91  | 12 |
|    |     |    |

# Strided convolution



|   |   |   |   |                 |                |                |
|---|---|---|---|-----------------|----------------|----------------|
| 2 | 3 | 7 | 4 | 6               | 2              | 9              |
| 6 | 6 | 9 | 8 | 7               | 4              | 3              |
| 3 | 4 | 8 | 3 | 8               | 9              | 7              |
| 7 | 8 | 3 | 6 | 6               | 3              | 4              |
| 4 | 2 | 1 | 8 | 3 <sup>3</sup>  | 4 <sup>4</sup> | 6 <sup>4</sup> |
| 3 | 2 | 4 | 1 | 9 <sup>1</sup>  | 8 <sup>0</sup> | 3 <sup>2</sup> |
| 0 | 1 | 3 | 9 | 2 <sup>-1</sup> | 1 <sup>0</sup> | 4 <sup>3</sup> |

7x7

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

3x3

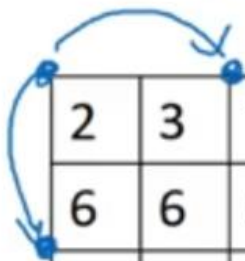
Stride = 2

=

|    |     |     |
|----|-----|-----|
| 91 | 100 | 83  |
| 69 | 91  | 127 |
| 44 | 72  | 74  |



# Strided convolution



|   |   |   |   |                 |                |                |
|---|---|---|---|-----------------|----------------|----------------|
| 2 | 3 | 7 | 4 | 6               | 2              | 9              |
| 6 | 6 | 9 | 8 | 7               | 4              | 3              |
| 3 | 4 | 8 | 3 | 8               | 9              | 7              |
| 7 | 8 | 3 | 6 | 6               | 3              | 4              |
| 4 | 2 | 1 | 8 | 3 <sup>3</sup>  | 4 <sup>4</sup> | 6 <sup>4</sup> |
| 3 | 2 | 4 | 1 | 9 <sup>1</sup>  | 8 <sup>0</sup> | 3 <sup>2</sup> |
| 0 | 1 | 3 | 9 | 2 <sup>-1</sup> | 1 <sup>0</sup> | 4 <sup>3</sup> |

7x7

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

3x3

stride = 2

=

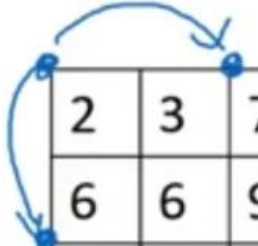
|    |     |     |
|----|-----|-----|
| 91 | 100 | 83  |
| 69 | 91  | 127 |
| 44 | 72  | 74  |

3x3

$n \times n$       \*       $f \times f$   
 padding  $p$       stride  $s$   
                           $s=2$

$$\frac{n+2p-f}{s} + 1 \quad \times \quad \frac{n+2p-f}{s} + 1$$

# Strided convolution



|   |   |   |   |                 |                |                |
|---|---|---|---|-----------------|----------------|----------------|
| 2 | 3 | 7 | 4 | 6               | 2              | 9              |
| 6 | 6 | 9 | 8 | 7               | 4              | 3              |
| 3 | 4 | 8 | 3 | 8               | 9              | 7              |
| 7 | 8 | 3 | 6 | 6               | 3              | 4              |
| 4 | 2 | 1 | 8 | 3 <sup>3</sup>  | 4 <sup>4</sup> | 6 <sup>4</sup> |
| 3 | 2 | 4 | 1 | 9 <sup>1</sup>  | 8 <sup>0</sup> | 3 <sup>2</sup> |
| 0 | 1 | 3 | 9 | 2 <sup>-1</sup> | 1 <sup>0</sup> | 4 <sup>3</sup> |

7x7

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

3x3

stride = 2

=

|    |     |     |
|----|-----|-----|
| 91 | 100 | 83  |
| 69 | 91  | 127 |
| 44 | 72  | 74  |

3x3

$\lfloor z \rfloor = \text{floor}(z)$

$n \times n$       \*       $f \times f$   
 padding  $p$       stride  $s$   
                           $s=2$

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

$$\frac{7+0-3}{2} + 1 = \frac{4}{2} + 1 = 3$$

Andrew Ng

The way this is implemented is that you take

# Strided convolution

|   |   |   |   |                 |                |                |
|---|---|---|---|-----------------|----------------|----------------|
| 2 | 3 | 7 | 4 | 6               | 2              | 9              |
| 6 | 6 | 9 | 8 | 7               | 4              | 3              |
| 3 | 4 | 8 | 3 | 8               | 9              | 7              |
| 7 | 8 | 3 | 6 | 6               | 3              | 4              |
| 4 | 2 | 1 | 8 | 3 <sup>3</sup>  | 4 <sup>4</sup> | 6 <sup>4</sup> |
| 3 | 2 | 4 | 1 | 9 <sup>1</sup>  | 8 <sup>0</sup> | 3 <sup>2</sup> |
| 0 | 1 | 3 | 9 | 2 <sup>-1</sup> | 1 <sup>0</sup> | 4 <sup>3</sup> |

7x7

|    |   |   |
|----|---|---|
| 3  | 4 | 4 |
| 1  | 0 | 2 |
| -1 | 0 | 3 |

3x3

stride = 2

=

|    |     |     |
|----|-----|-----|
| 91 | 100 | 83  |
| 69 | 91  | 127 |
| 44 | 72  | 74  |

3x3

$\lfloor z \rfloor = \text{floor}(z)$

$n \times n$  \*  $f \times f$   
 padding  $p$       stride  $s$   
 $s = 2$

$$\left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n + 2p - f}{s} + 1 \right\rfloor$$

$$\frac{7 + 0 - 3}{2} + 1 = \frac{4}{2} + 1 = 3$$

Andrew Ng

Then it turns out that if that's the convention that your three by three filter,

# Summary of convolutions

$n \times n$  image       $f \times f$  filter

padding  $p$       stride  $s$

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

# Technical note on cross-correlation vs. convolution

Convolution in math textbook:

|                |                |                |   |   |   |
|----------------|----------------|----------------|---|---|---|
| 2 <sup>7</sup> | 3 <sup>2</sup> | 7 <sup>5</sup> | 4 | 6 | 2 |
| 6 <sup>9</sup> | 6 <sup>0</sup> | 9 <sup>4</sup> | 8 | 7 | 4 |
| 3 <sup>7</sup> | 4 <sup>1</sup> | 8 <sup>3</sup> | 3 | 8 | 9 |
| 7              | 8              | 3              | 6 | 6 | 3 |
| 4              | 2              | 1              | 8 | 3 | 4 |
| 3              | 2              | 4              | 1 | 9 | 8 |

|  |    |   |   |
|--|----|---|---|
|  | 3  | 4 | 5 |
|  | 1  | 0 | 2 |
|  | -1 | 9 | 7 |

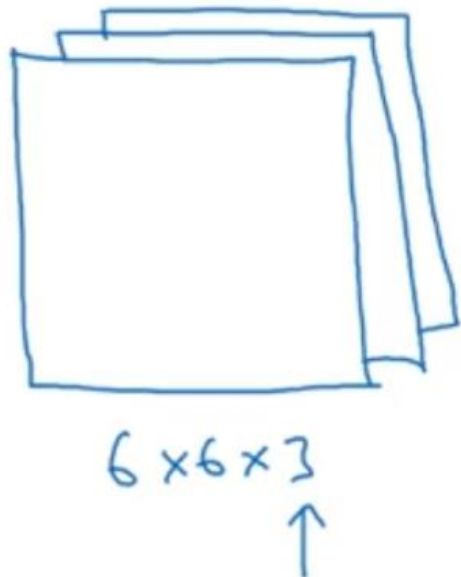
  

|    |   |   |
|----|---|---|
| 7  | 2 | 5 |
| 9  | 0 | 4 |
| -1 | 1 | 3 |

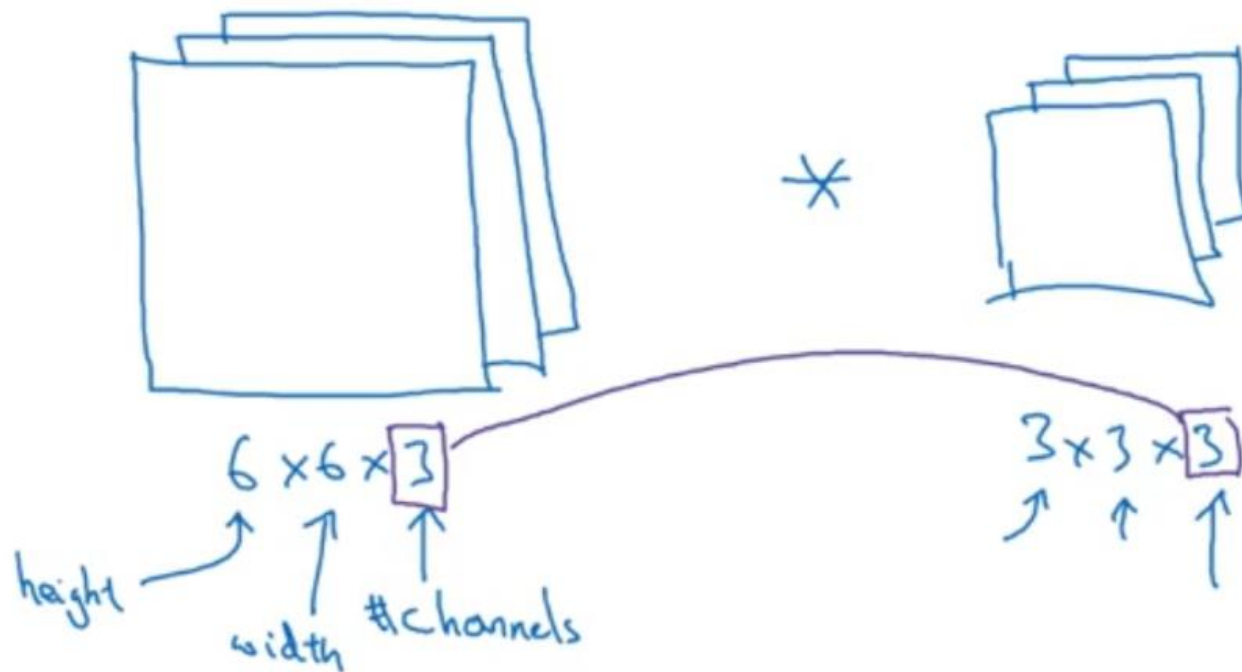
|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

$$(A * B) * C = A * (B * C)$$

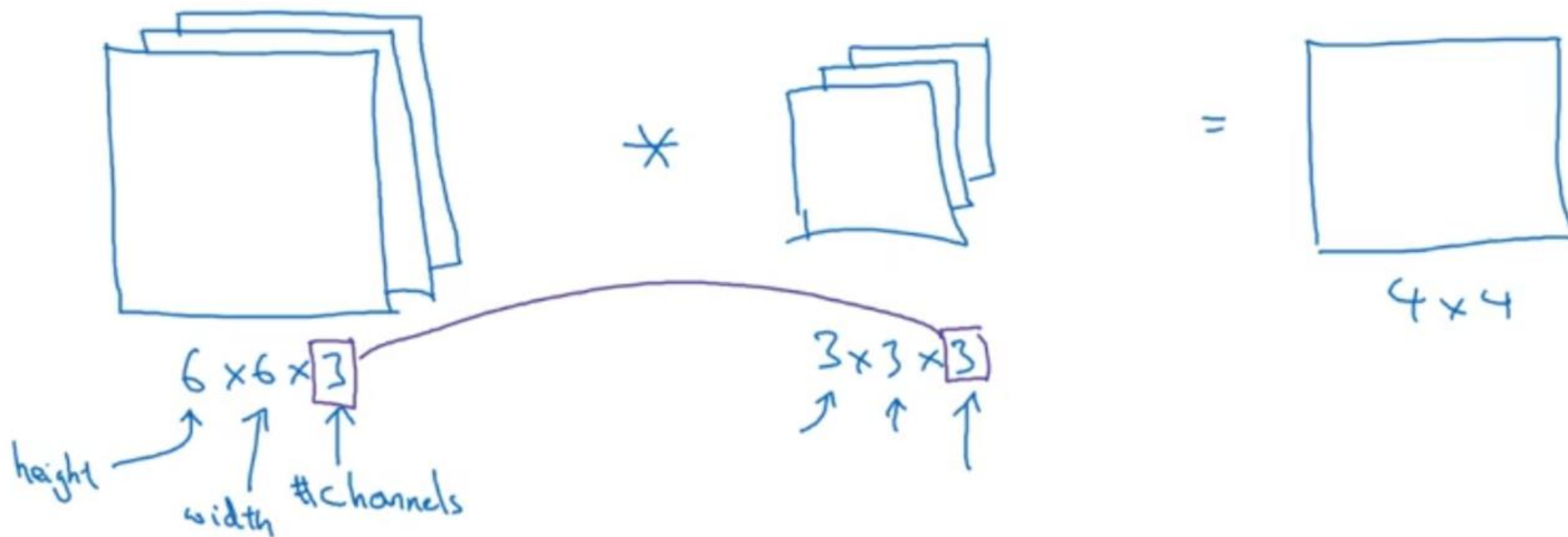
# Convolutions on RGB images



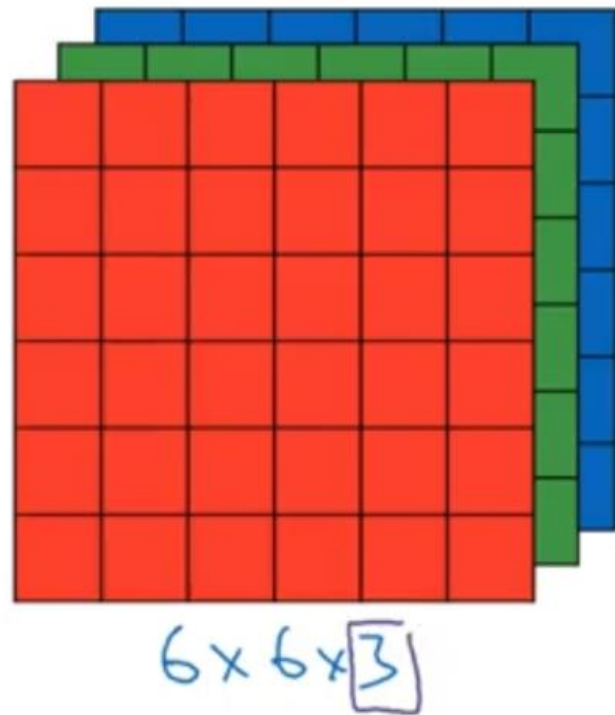
# Convolutions on RGB images



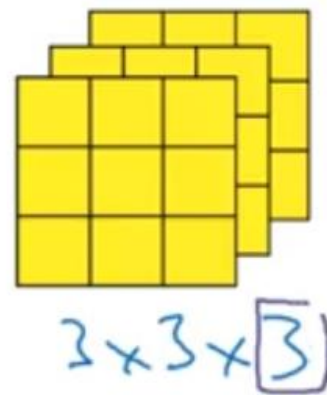
# Convolutions on RGB images



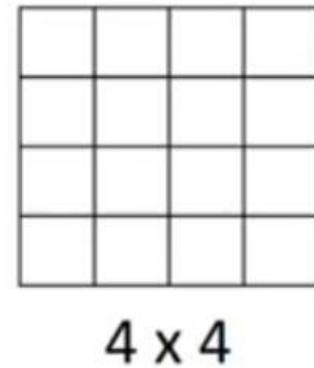
# Convolutions on RGB image



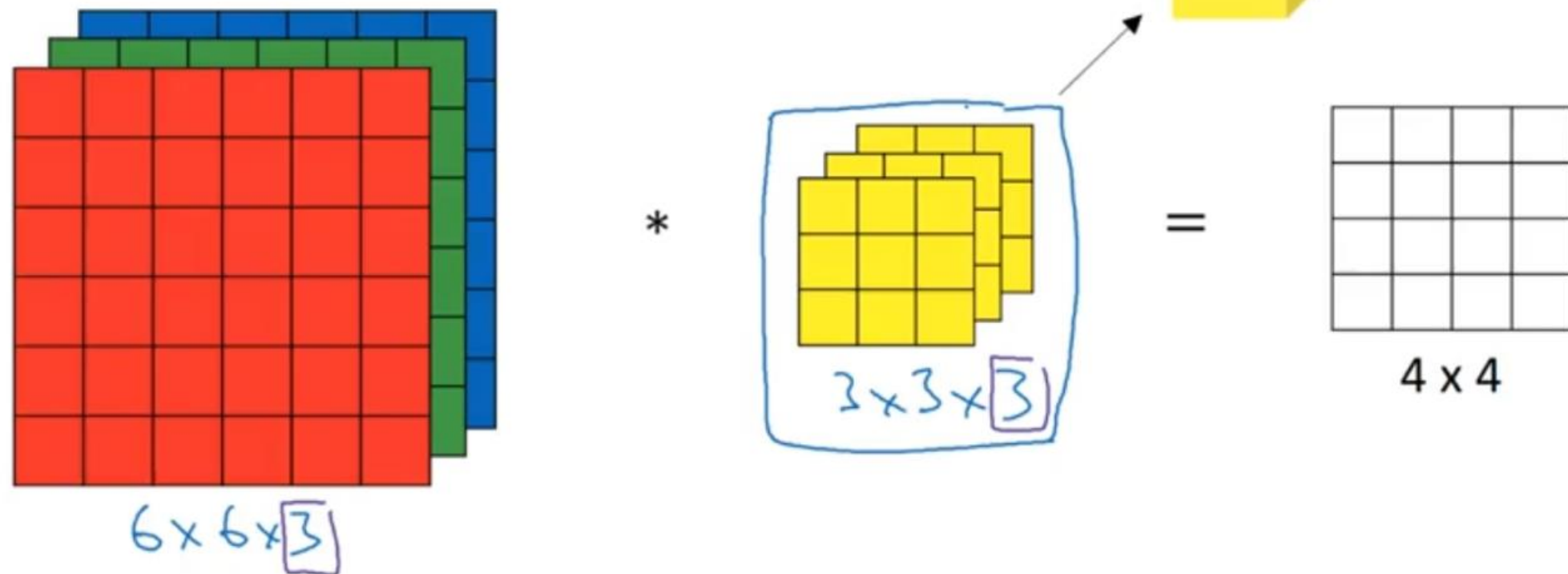
\*



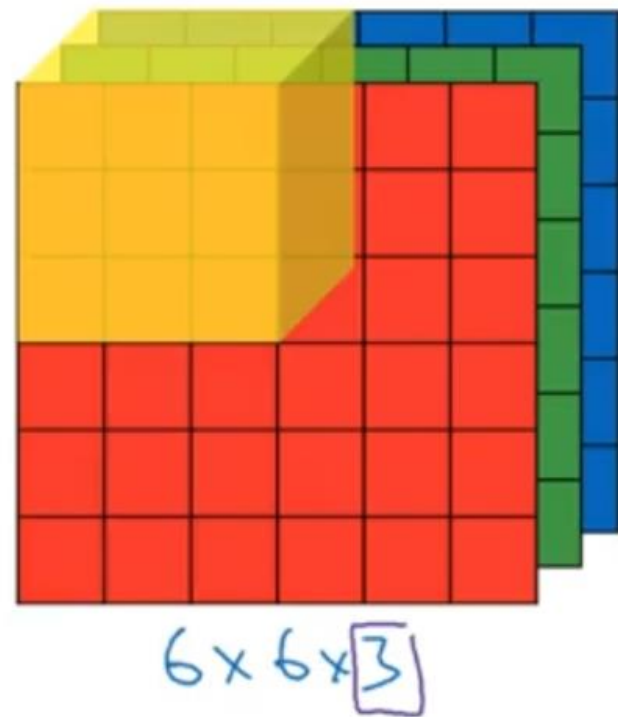
=



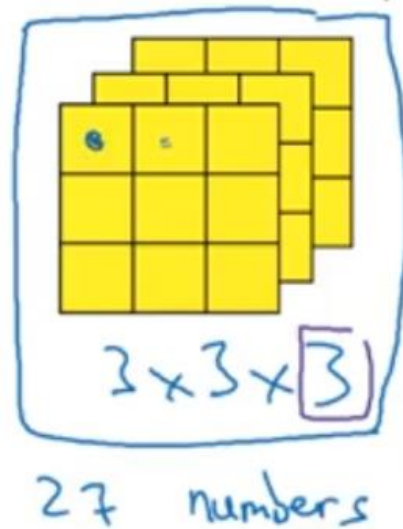
# Convolutions on RGB image



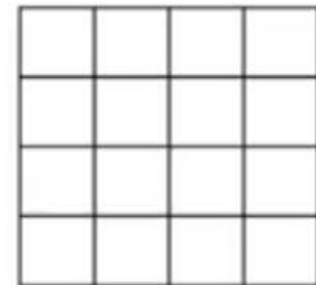
# Convolutions on RGB image



\*

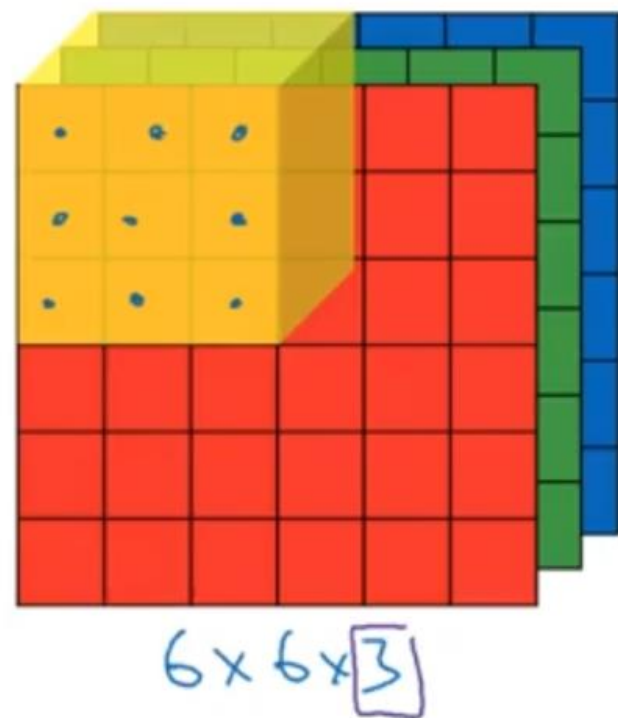


=

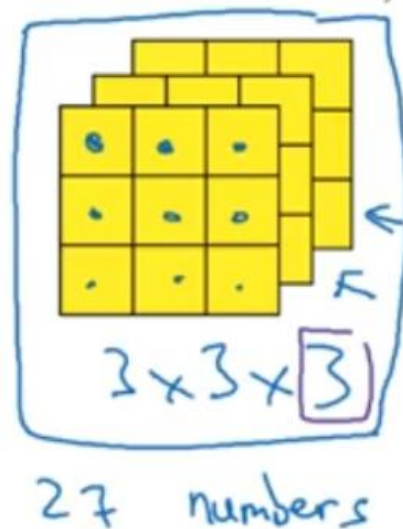


$4 \times 4$

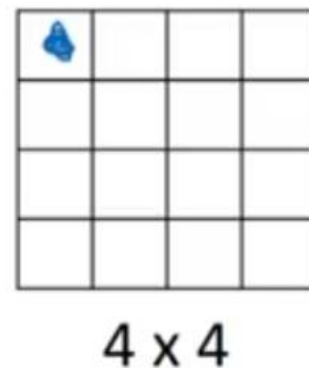
# Convolutions on RGB image



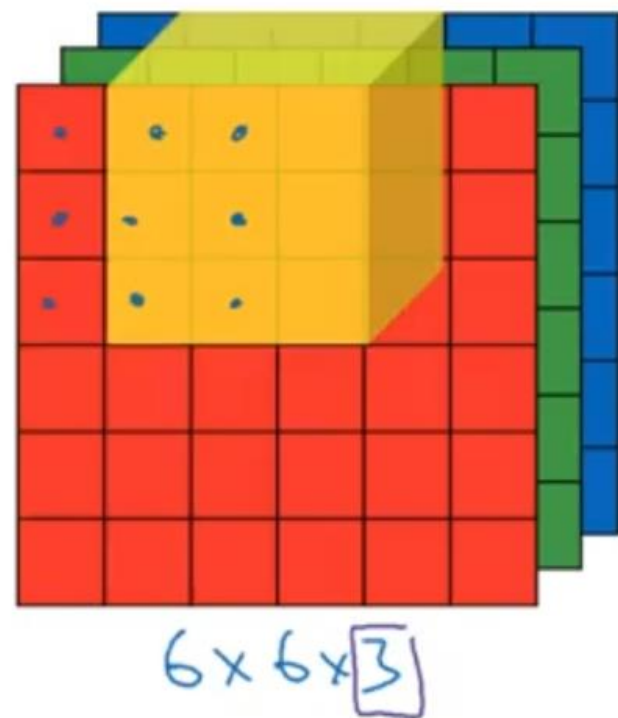
\*



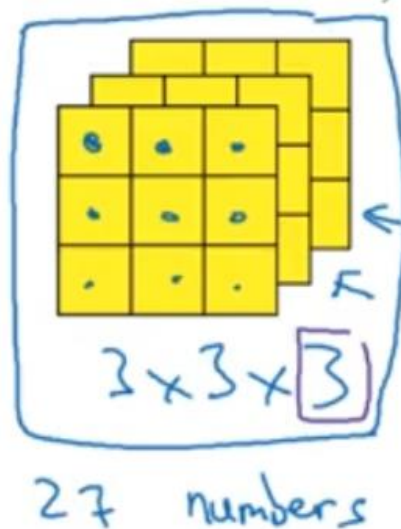
=



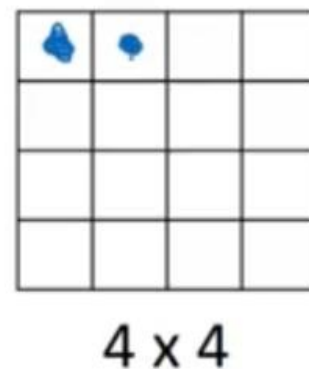
# Convolutions on RGB image



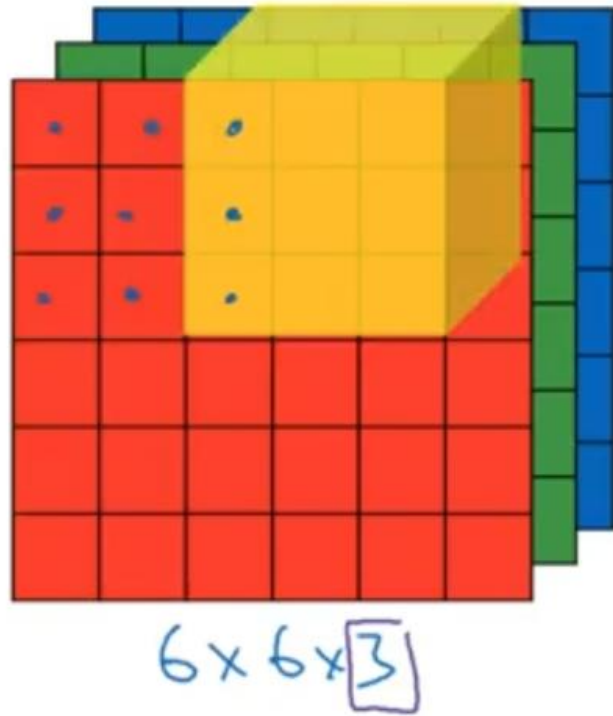
\*



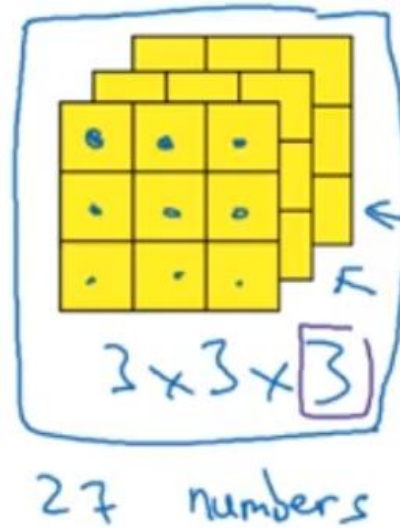
=



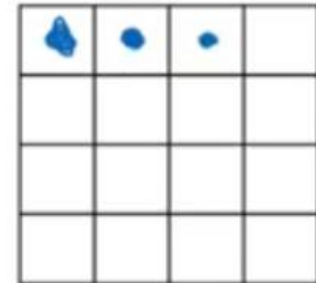
# Convolutions on RGB image



\*



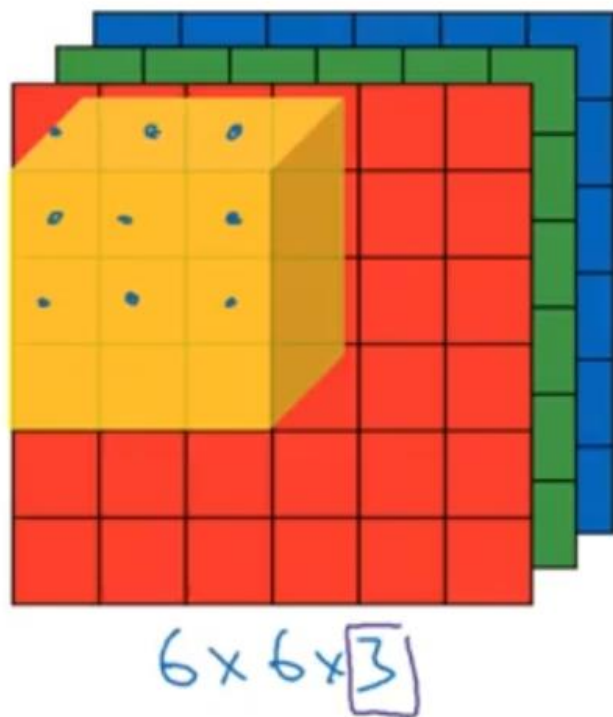
=



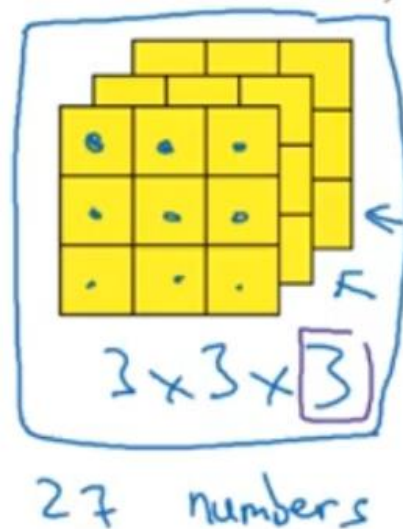
4x4



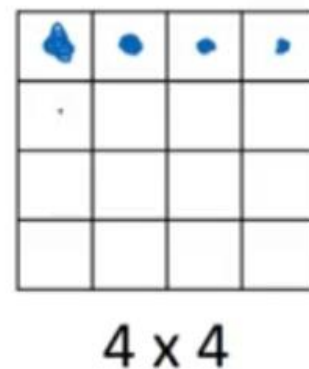
# Convolutions on RGB image



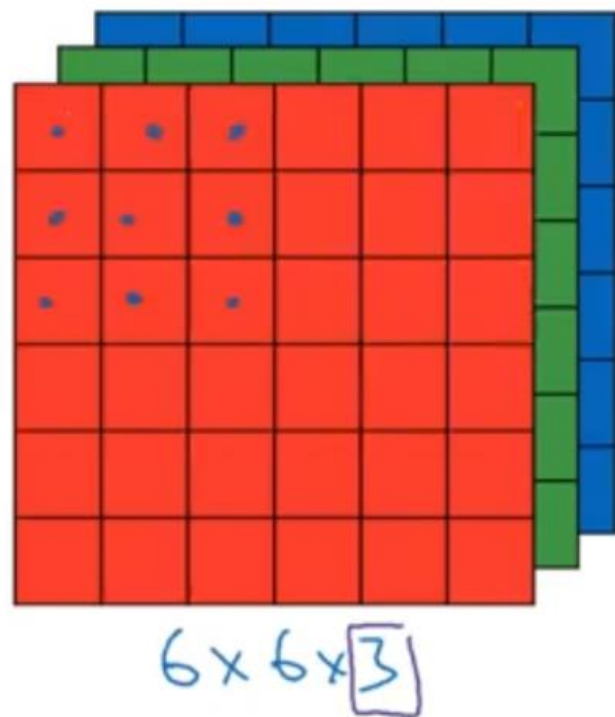
\*



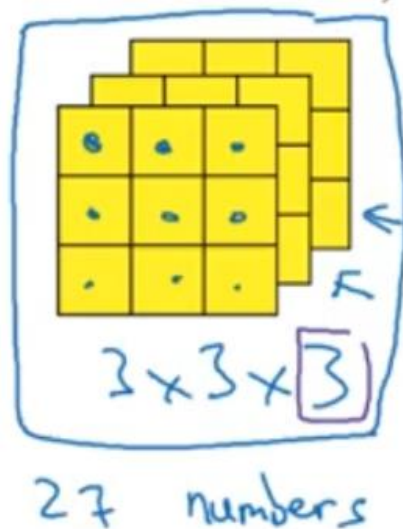
=



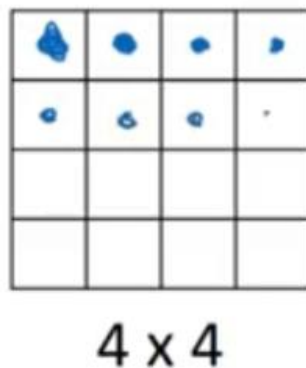
# Convolutions on RGB image



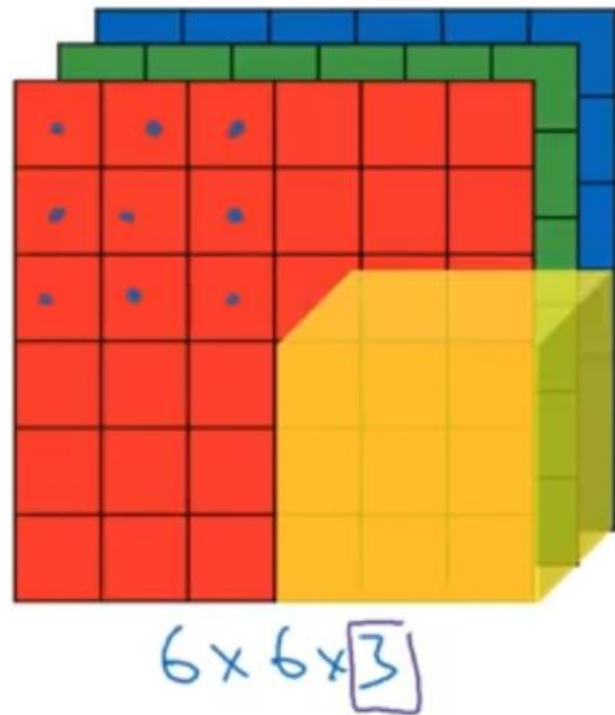
\*



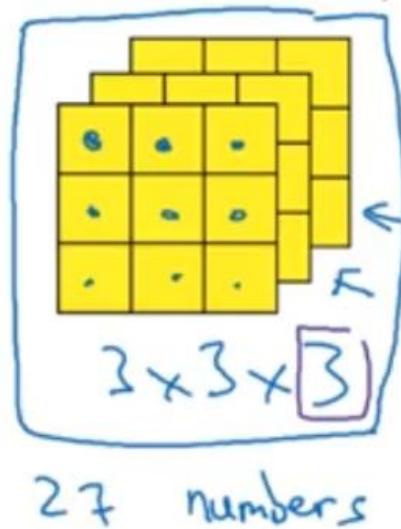
=



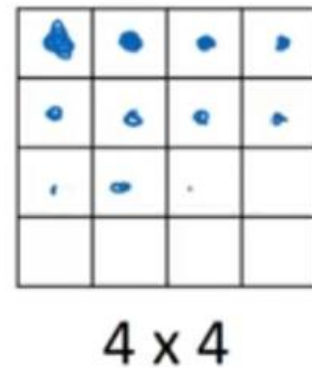
# Convolutions on RGB image



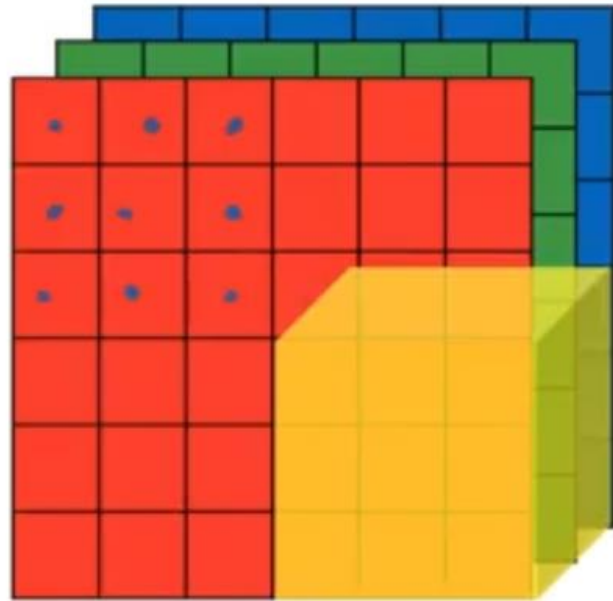
\*



=

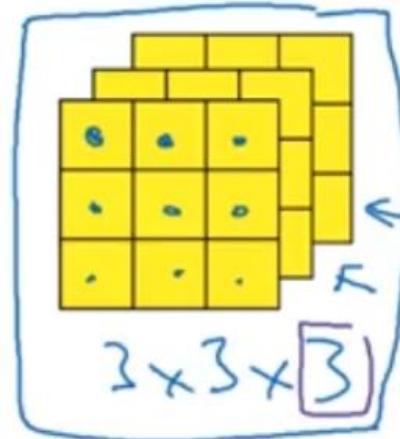


# Convolutions on RGB image



$6 \times 6 \times 3$

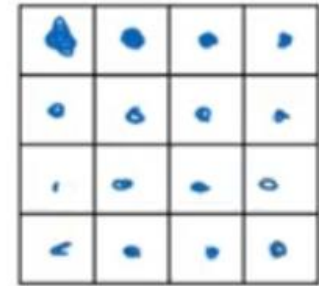
\*



$3 \times 3 \times 3$

27 numbers

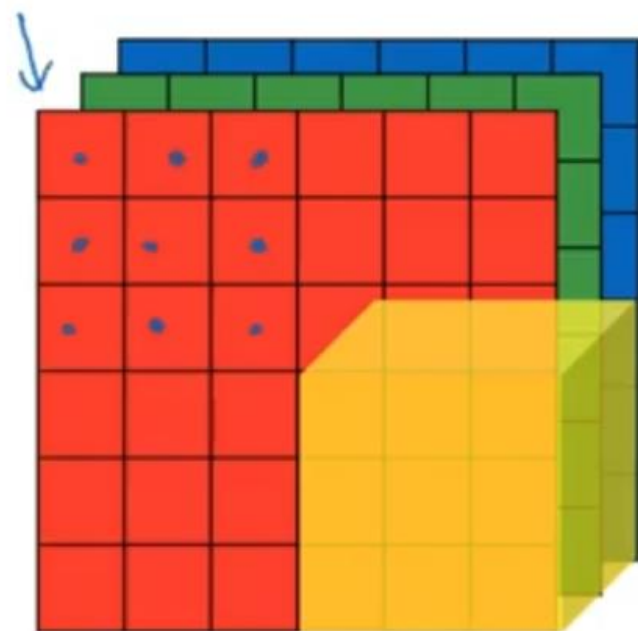
=



$4 \times 4$

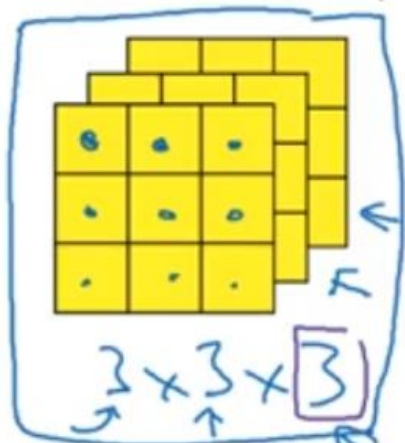


# Convolutions on RGB image



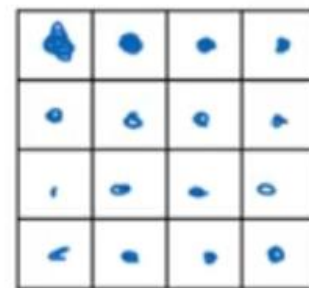
$6 \times 6 \times 3$   
 $\uparrow \quad \uparrow \quad \uparrow$

\*



27 numbers

=

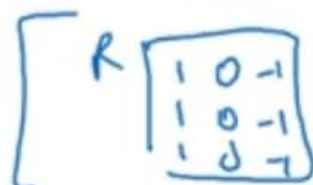


$4 \times 4$

$\begin{bmatrix} R \\ G \\ B \end{bmatrix}$



$\rightarrow 3 \times 3 \times 3$

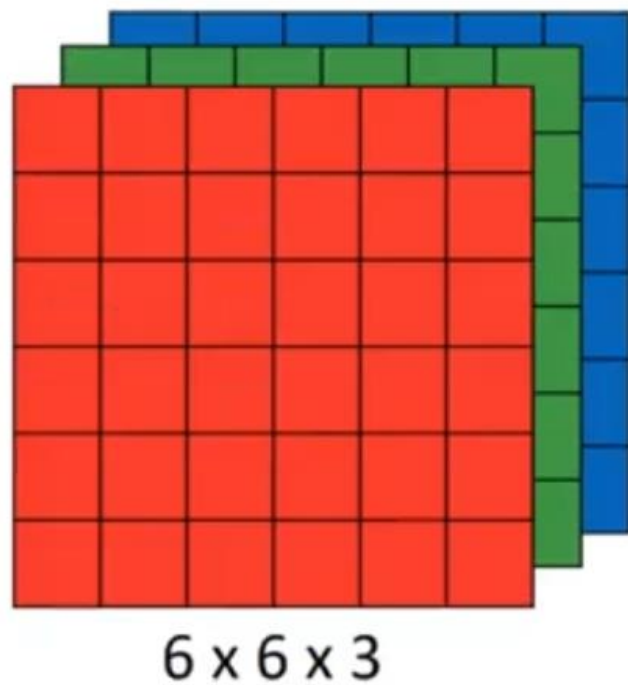


$\rightarrow 3 \times 3 \times 3$

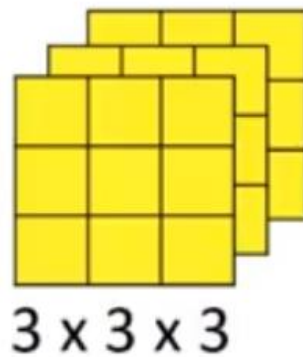
Andrew Ng

or maybe a filter looks at only the green channel and a blue channel.

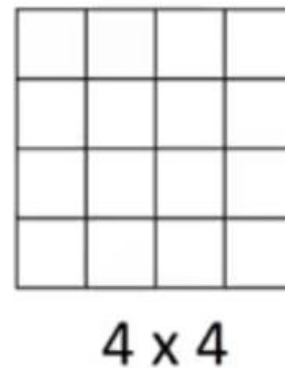
# Multiple filters



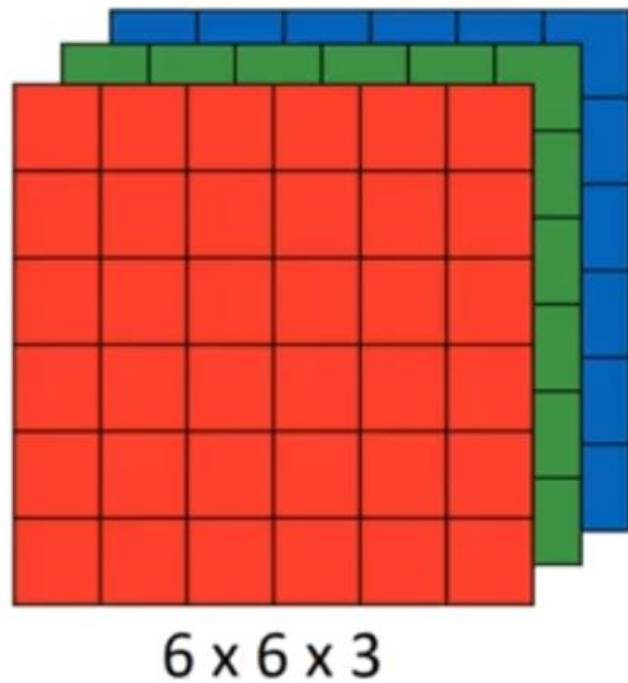
\*



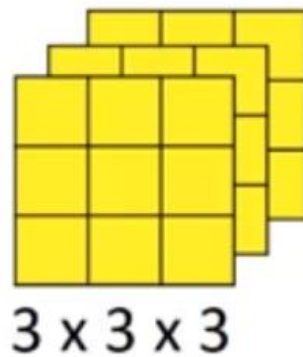
=



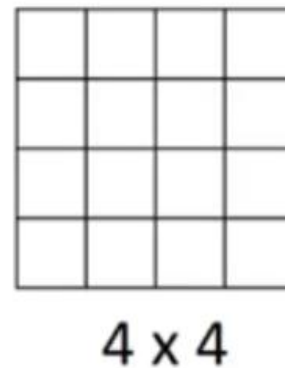
# Multiple filters



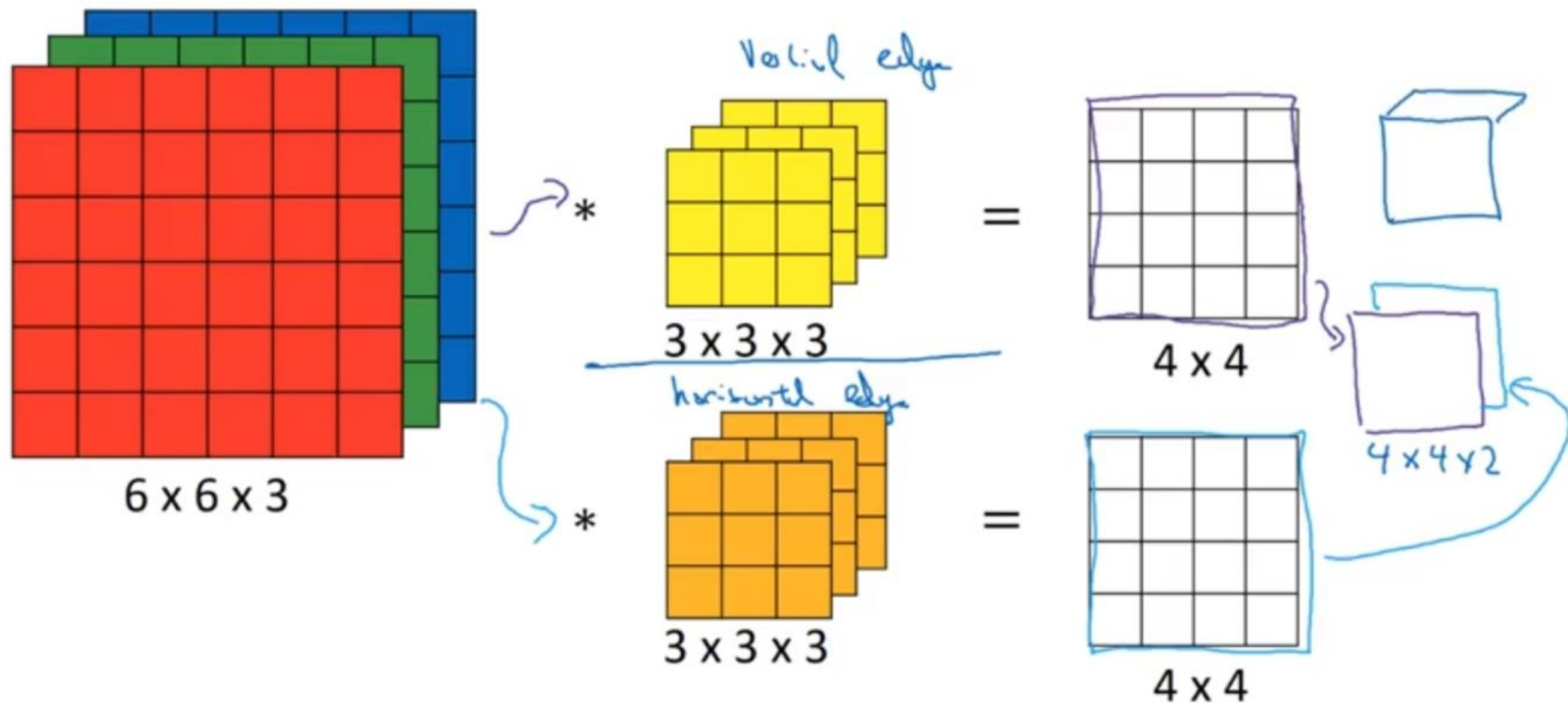
\*



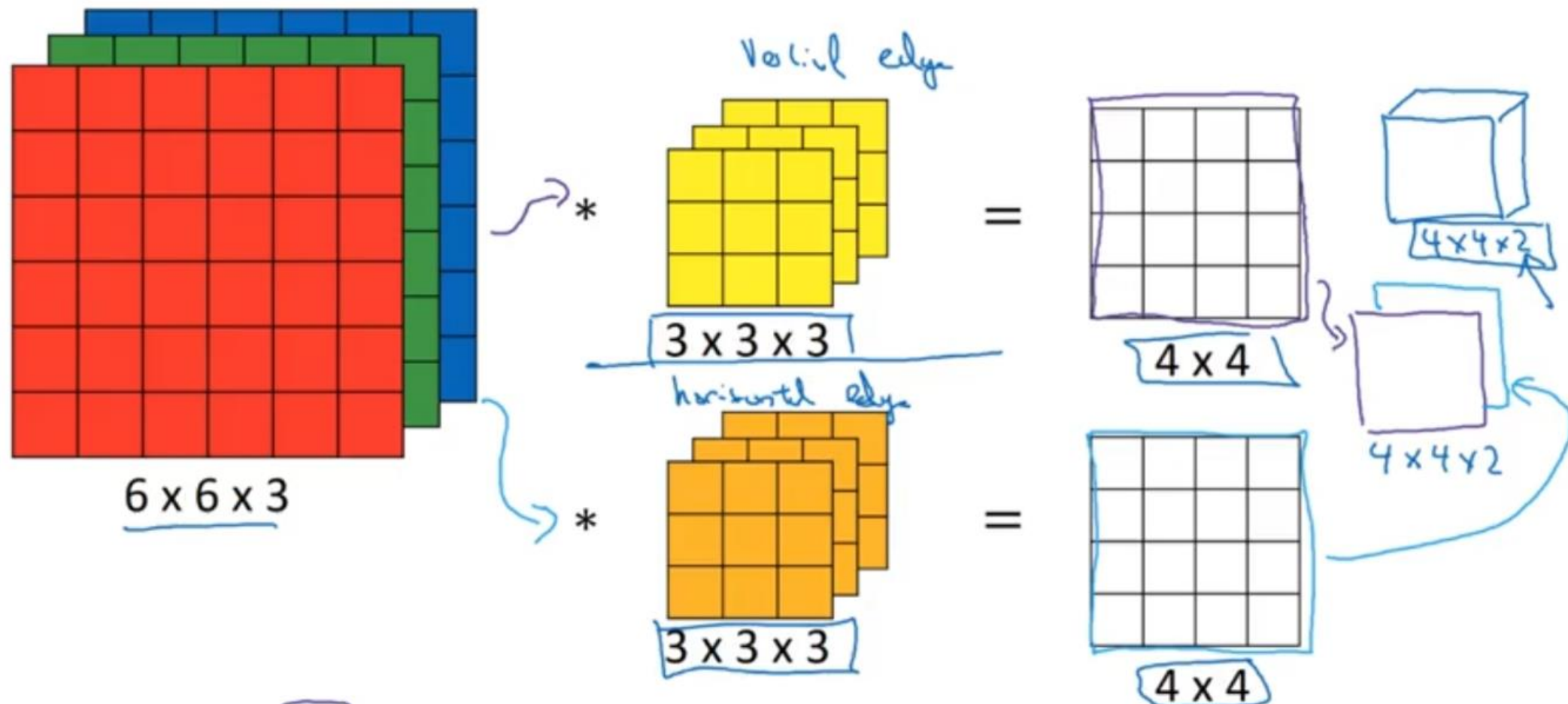
=



# Multiple filters



# Multiple filters

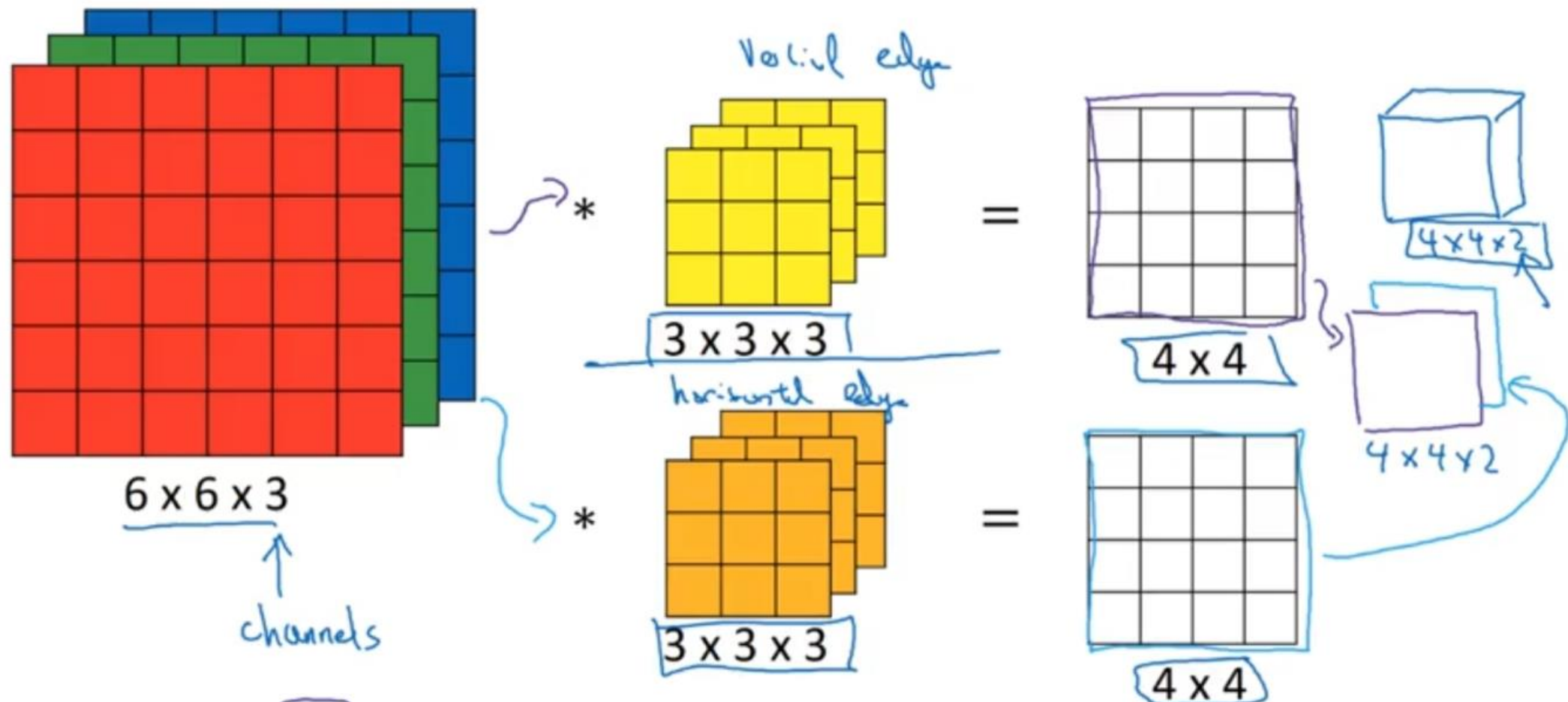


Summary:  $n \times n \times n_c$   $*$   $f \times f \times n_c$   $\rightarrow$   $n-f+1 \times n-f+1 \times n'_c$   
 $6 \times 6 \times 3$   $3 \times 3 \times 3$   $4 \times 4 \times 2$   $\uparrow$  #filters

Andrew Ng

And I wrote this assuming that you use a stride of one and no padding.

# Multiple filters



Summary:  $n \times n \times n_c$   $\times$   $f \times f \times n_c$   $\rightarrow$   $\frac{n-f+1}{4} \times \frac{n-f+1}{4} \times \frac{n_c}{2}$

$6 \times 6 \times 3$   $3 \times 3 \times 3$   $4 \times 4 \times 2$  #filters

Andrew Ng

the size of this third dimension of these filters.



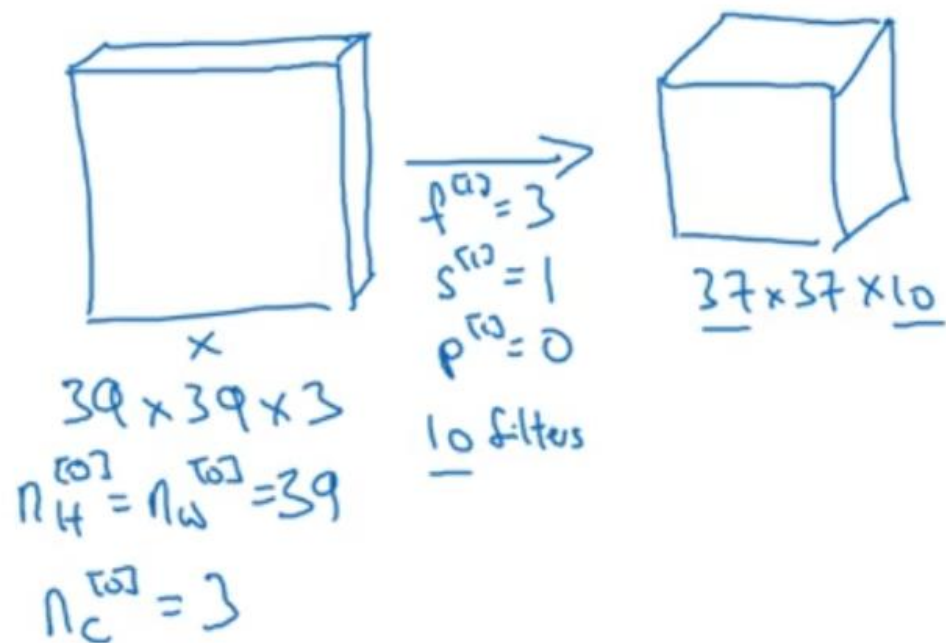
deeplearning.ai

# Convolutional Neural Networks

---

## A simple convolution network example

# Example ConvNet

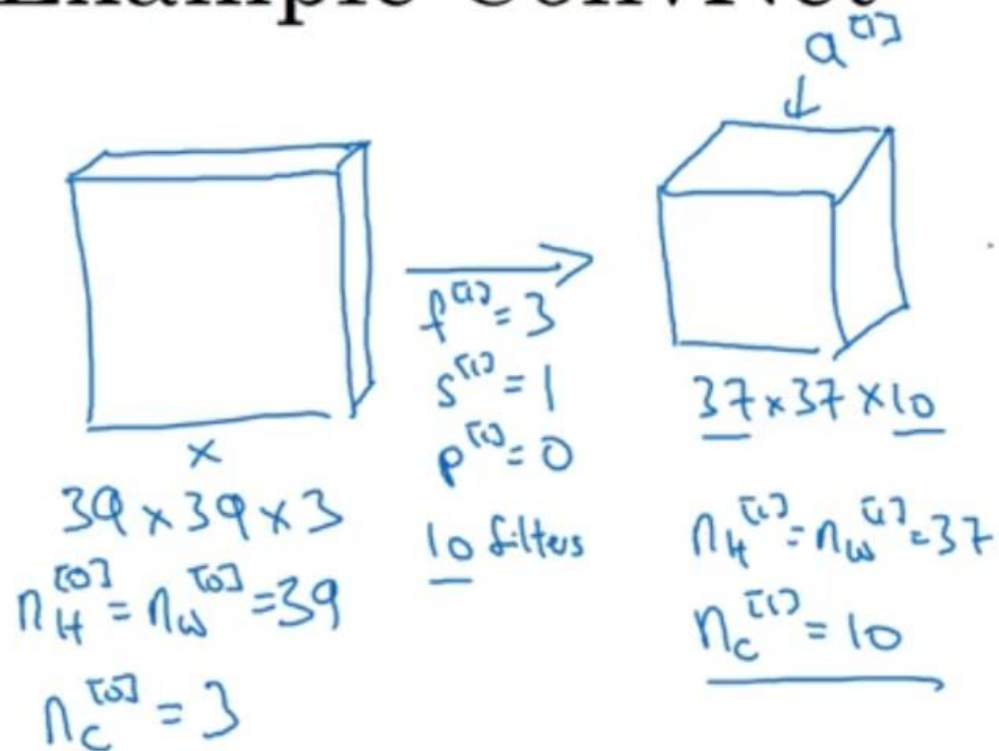


$$\frac{n+2p-f}{s} + 1$$

$$\frac{39+0-3}{1} + 1 = 37$$

So that's why the output is 37 by 37,  
it's a valid convolution and

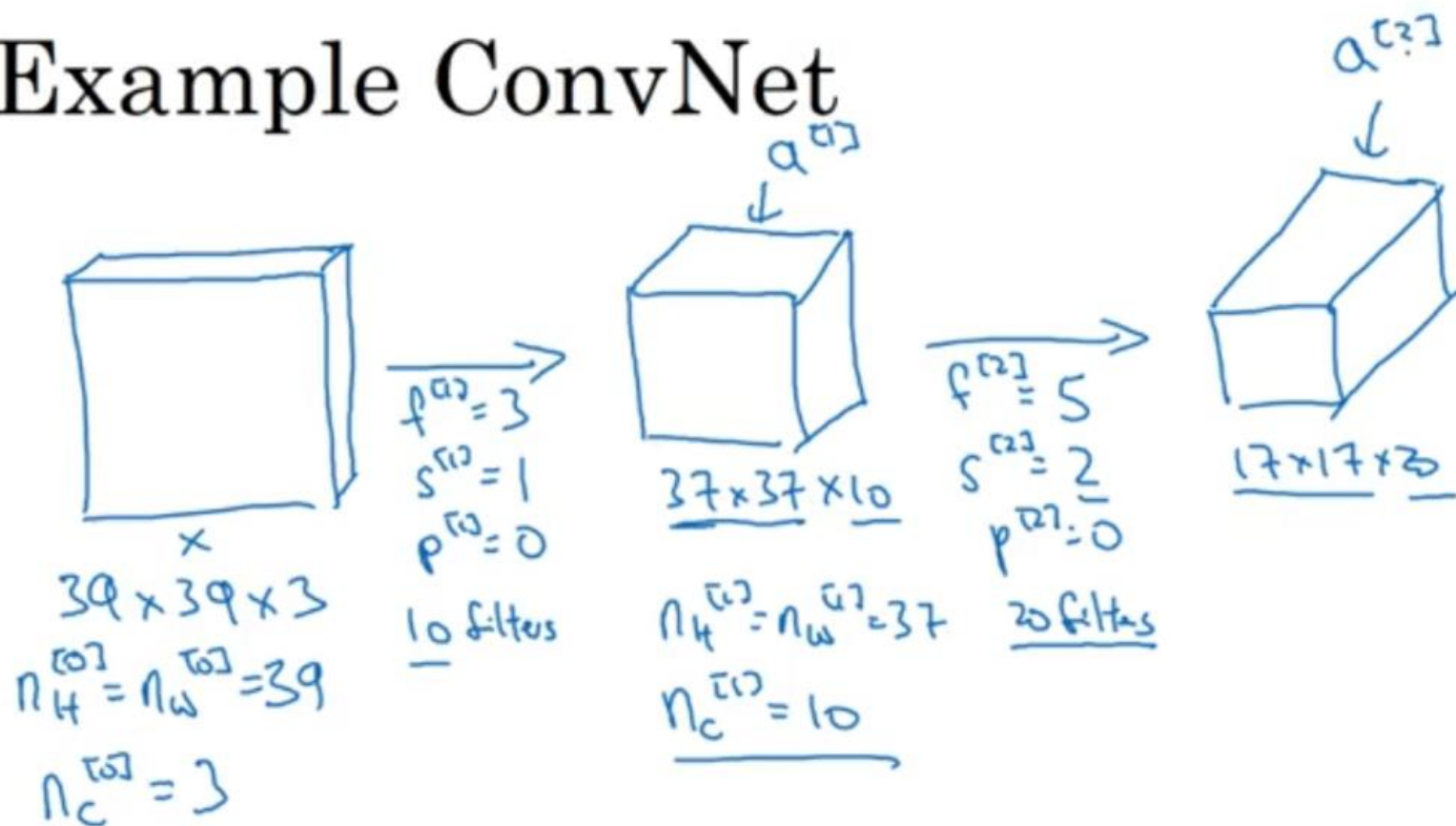
# Example ConvNet



$$\frac{n+2p-f}{s} + 1$$

$$\frac{39+0-3}{1} + 1 = 37$$

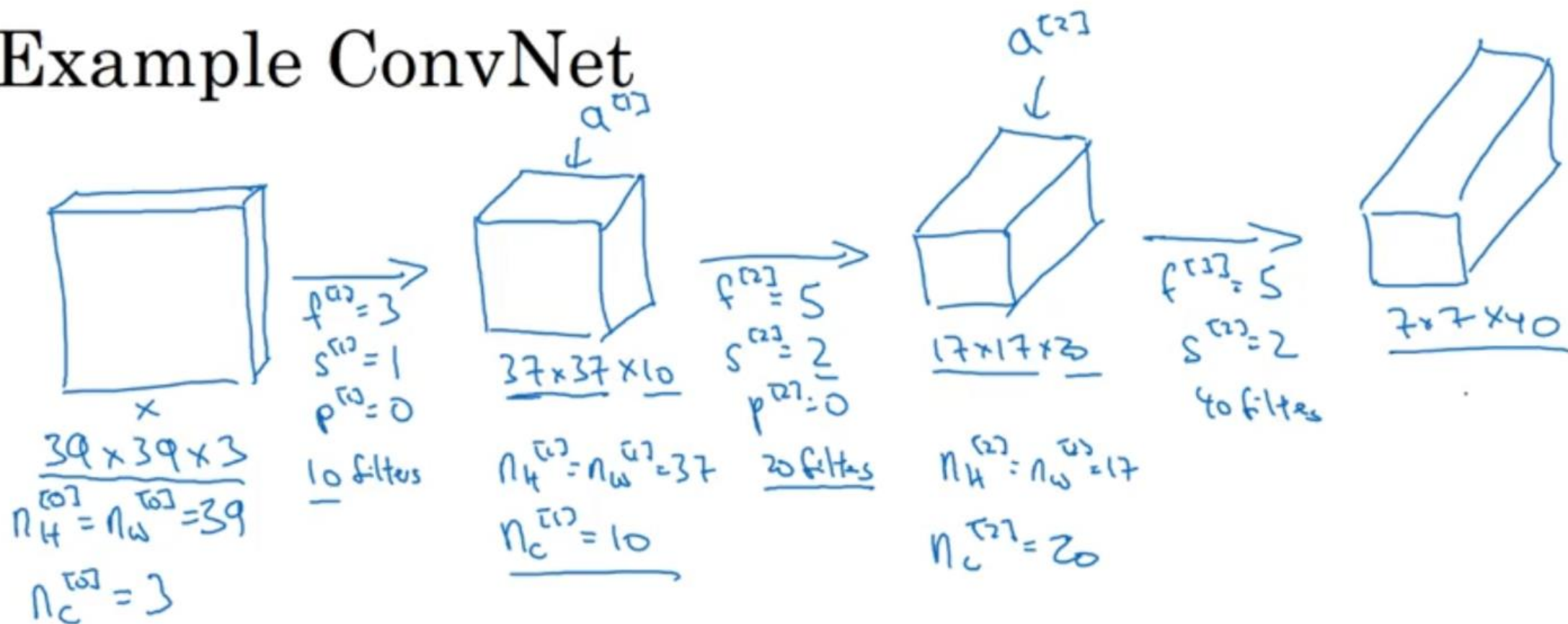
# Example ConvNet



$$\frac{n + 2p - f}{s} + 1$$

$$\frac{39 + 0 - 3}{1} + 1 = 37$$

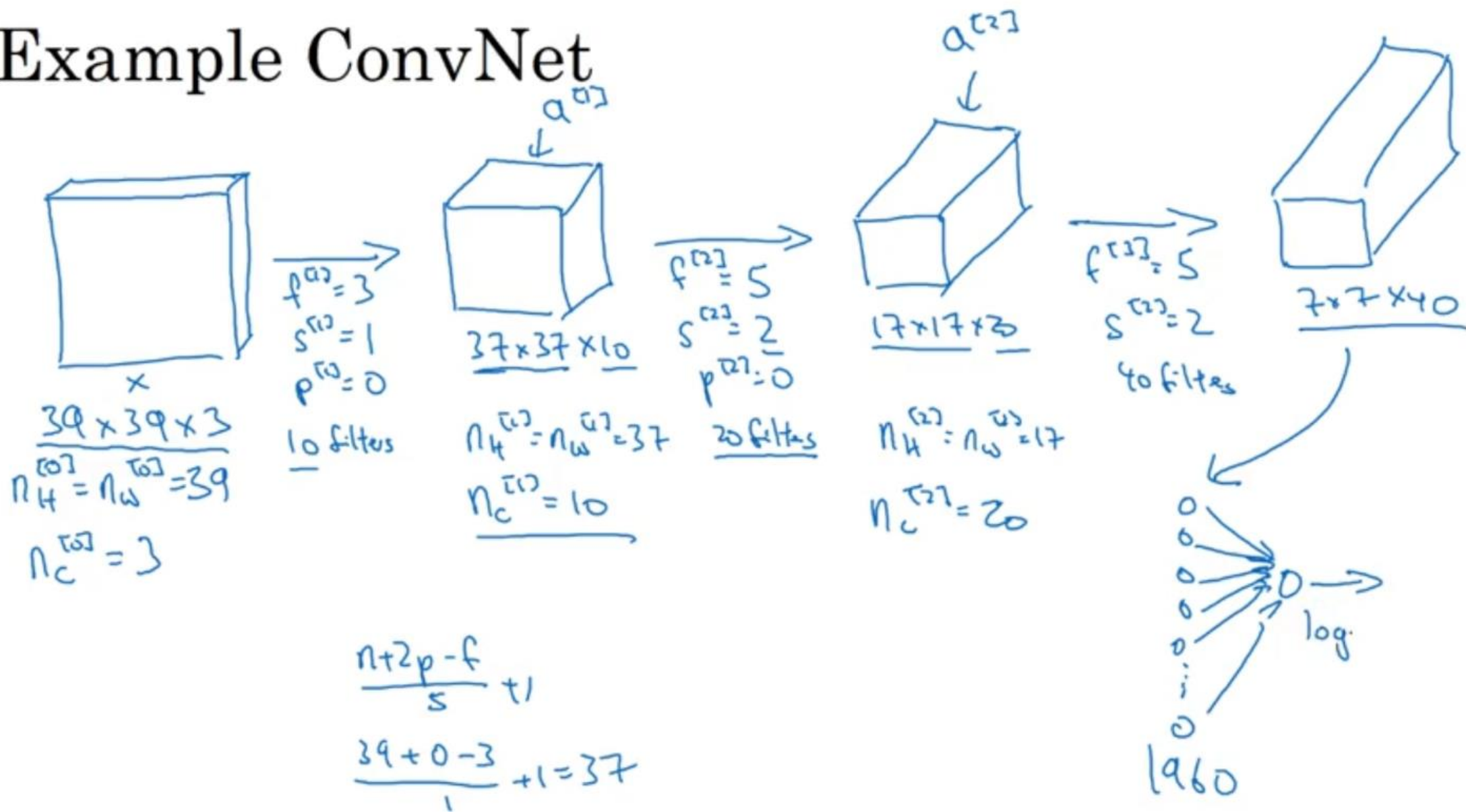
# Example ConvNet



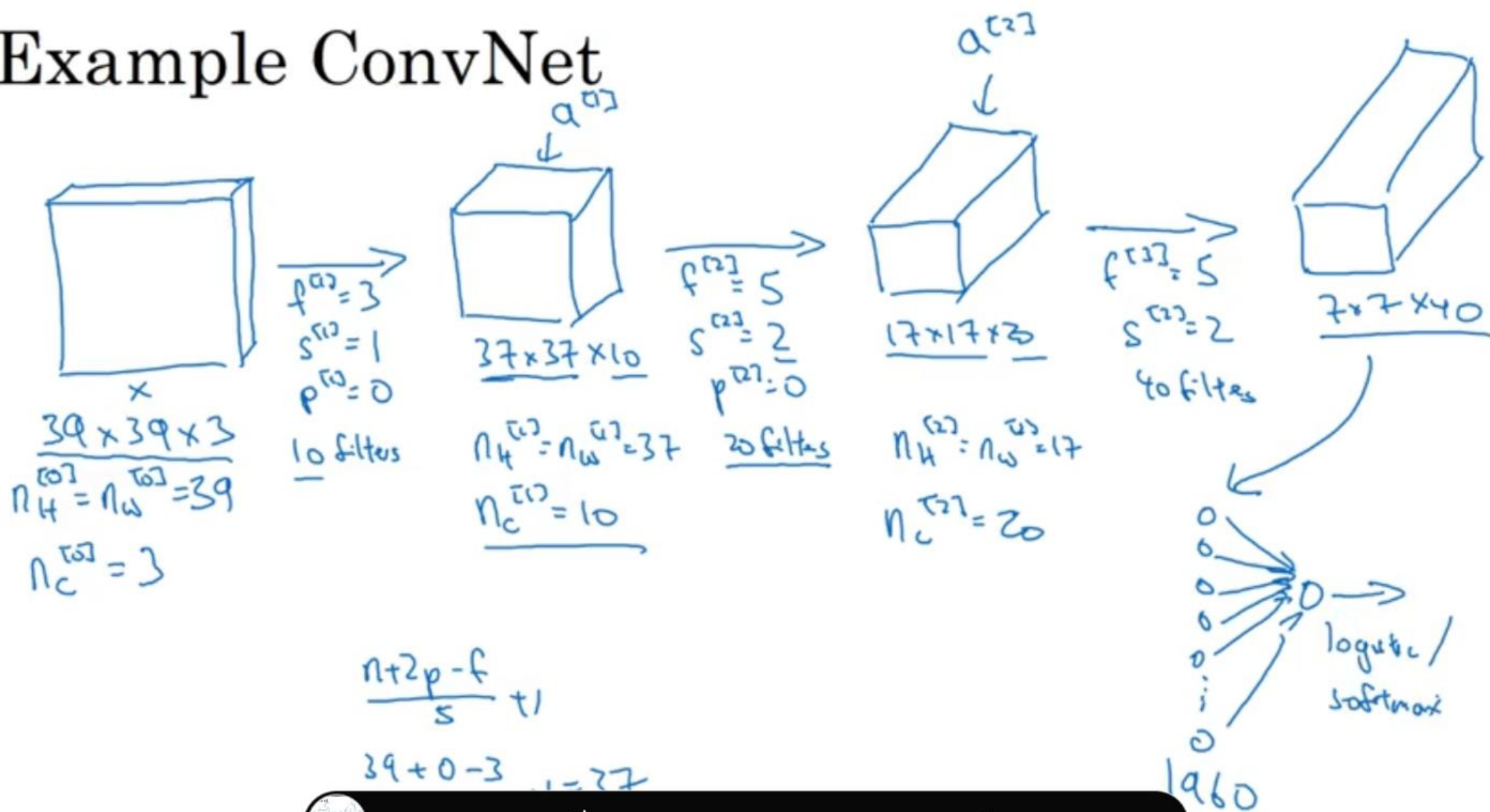
$$\frac{n + 2p - f}{s} + 1$$

$$\frac{39 + 0 - 3}{1} + 1 = 37$$

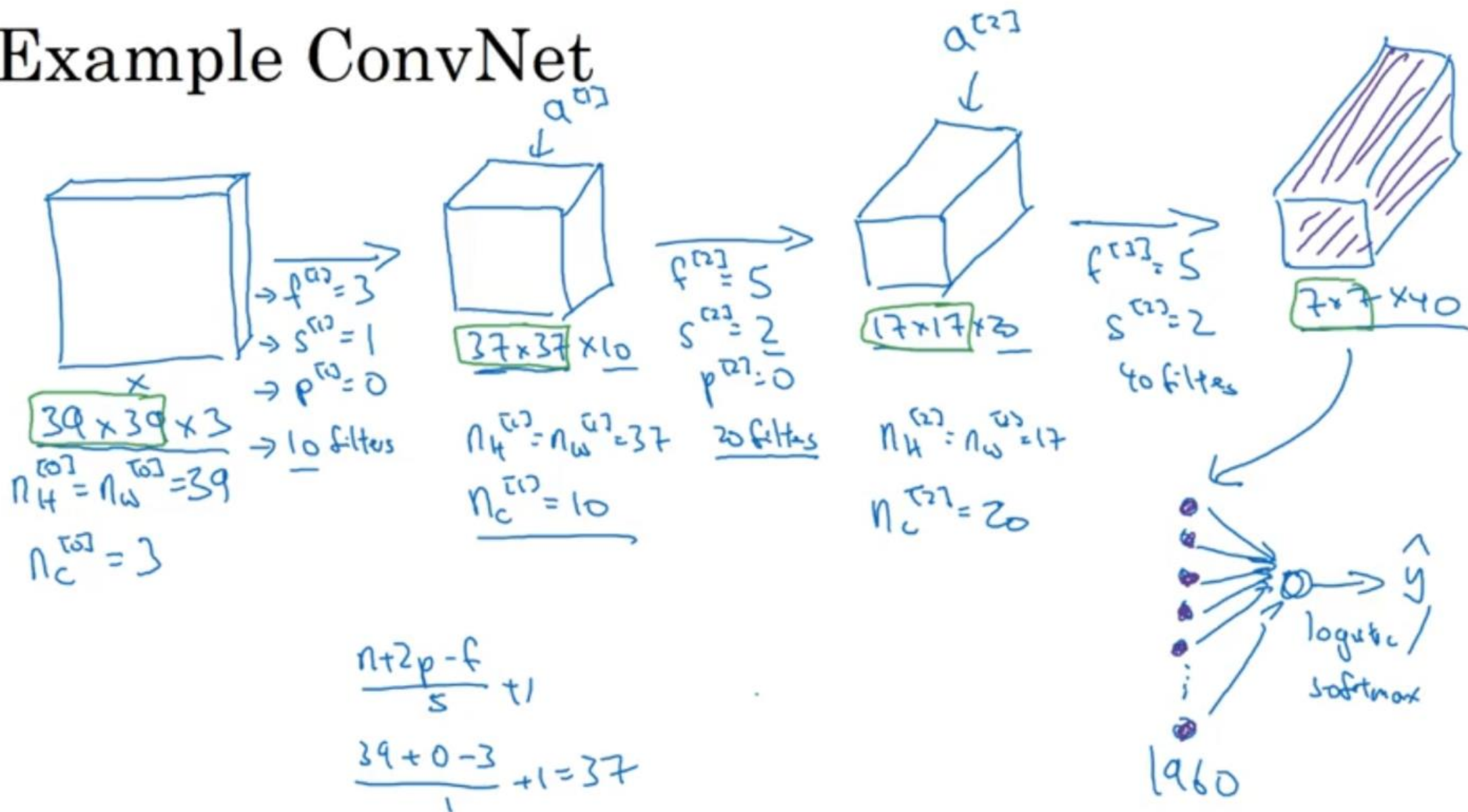
# Example ConvNet



# Example ConvNet



# Example ConvNet



# Types of layer in a convolutional network:

- Convolution (conv)
- Pooling (pool)
- Fully connected (FC)

# Pooling layer: Max pooling

|   |   |   |   |
|---|---|---|---|
| 1 | 3 | 2 | 1 |
| 2 | 9 | 1 | 1 |
| 1 | 3 | 2 | 3 |
| 5 | 6 | 1 | 2 |

4x4



|  |  |
|--|--|
|  |  |
|  |  |

# Pooling layer: Max pooling

|   |   |   |   |
|---|---|---|---|
| 1 | 3 | 2 | 1 |
| 2 | 9 | 1 | 1 |
| 1 | 3 | 2 | 3 |
| 5 | 6 | 1 | 2 |

4x4



|   |   |
|---|---|
| 9 | 2 |
| 6 | 3 |

2x2

# Pooling layer: Max pooling

|   |   |   |   |
|---|---|---|---|
| 1 | 3 | 2 | 1 |
| 2 | 9 | 1 | 1 |
| 1 | 3 | 2 | 3 |
| 5 | 6 | 1 | 2 |

4x4



|   |   |
|---|---|
| 9 | 2 |
| 6 | 3 |

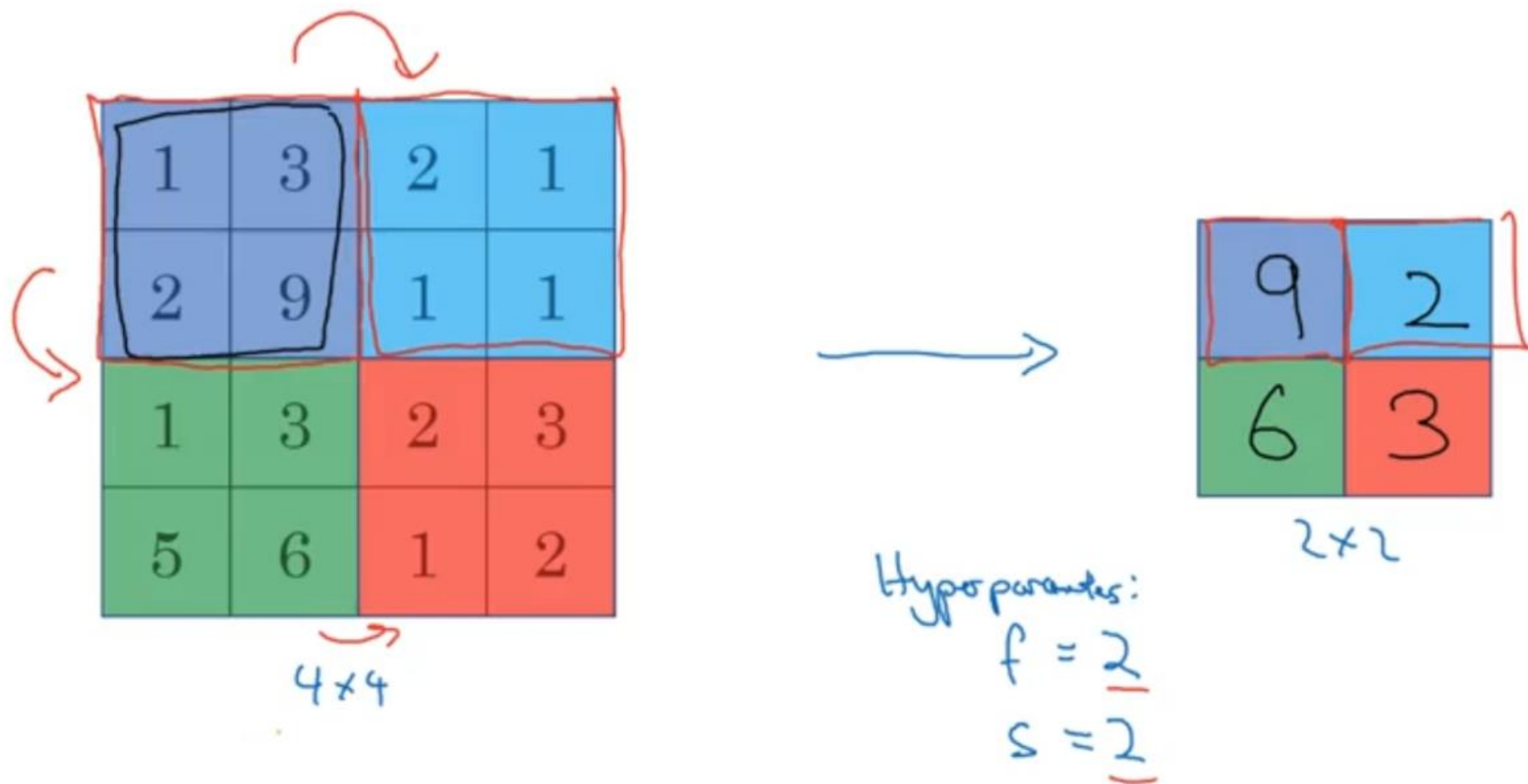
2x2

Hyper

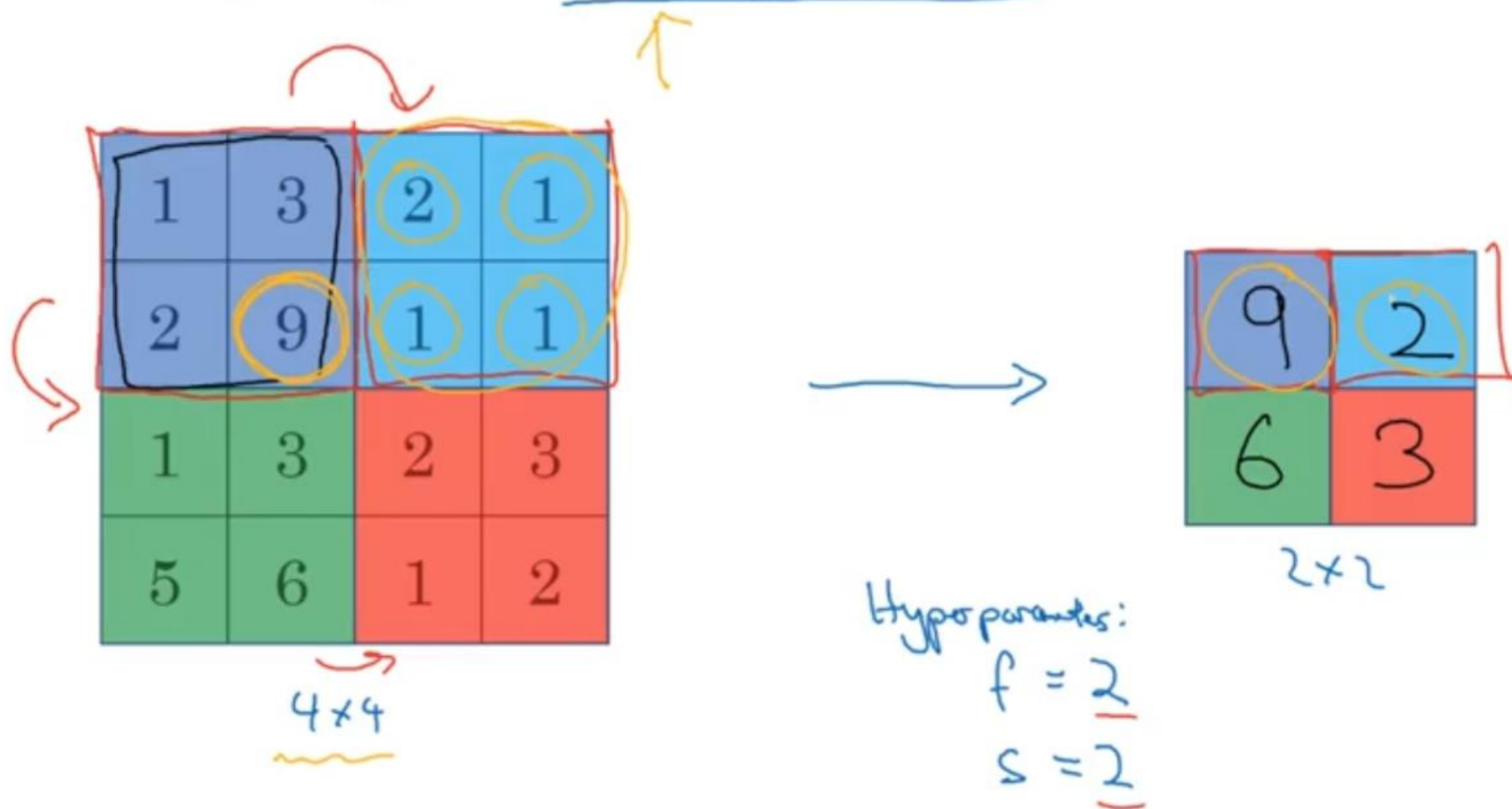
$$f = 2$$

$$s = 2$$

# Pooling layer: Max pooling



# Pooling layer: Max pooling



# Pooling layer: Max pooling

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 3 | 2 | 1 | 3 |
| 2 | 9 | 1 | 1 | 5 |
| 1 | 3 | 2 | 3 | 2 |
| 8 | 3 | 5 | 1 | 0 |
| 5 | 6 | 1 | 2 | 9 |

5x5

|  |  |  |
|--|--|--|
|  |  |  |
|  |  |  |
|  |  |  |

$$f = 3$$

$$s = 1$$

$$\left( \frac{n + 2p - f}{s} + 1 \right)$$

# Pooling layer: Max pooling



|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 3 | 2 | 1 | 3 |
| 2 | 9 | 1 | 1 | 5 |
| 1 | 3 | 2 | 3 | 2 |
| 8 | 3 | 5 | 1 | 0 |
| 5 | 6 | 1 | 2 | 9 |

5x5

|   |   |   |
|---|---|---|
| 9 | 9 | 5 |
|   |   |   |
|   |   |   |

$$\begin{array}{l} f=3 \\ \hline s=1 \end{array} \quad 3 \times 3$$

$$\left( \frac{n + 2p - f}{s} + 1 \right)$$

# Pooling layer: Max pooling



|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 3 | 2 | 1 | 3 |
| 2 | 9 | 1 | 1 | 5 |
| 1 | 3 | 2 | 3 | 2 |
| 8 | 3 | 5 | 1 | 0 |
| 5 | 6 | 1 | 2 | 9 |

5x5

|   |   |   |
|---|---|---|
| 9 | 9 | 5 |
| 9 | . |   |
|   |   |   |

3x3

$$\frac{f=3}{s=1}$$

$$1 \times 1 + 2 \times p - f + 1$$



# Pooling layer: Max pooling

↻

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 3 | 2 | 1 | 3 |
| 2 | 9 | 1 | 1 | 5 |
| 1 | 3 | 2 | 3 | 2 |
| 8 | 3 | 5 | 1 | 0 |
| 5 | 6 | 1 | 2 | 9 |

5x5

|   |   |   |
|---|---|---|
| 9 | 9 | 5 |
| 9 | 9 | 5 |
|   |   |   |

$$\begin{array}{l} f=3 \\ \hline s=1 \end{array} \quad 3 \times 3$$

$$\left( \frac{n + 2p - f}{s} + 1 \right)$$

# Pooling layer: Max pooling



|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 3 | 2 | 1 | 3 |
| 2 | 9 | 1 | 1 | 5 |
| 1 | 3 | 2 | 3 | 2 |
| 8 | 3 | 5 | 1 | 0 |
| 5 | 6 | 1 | 2 | 9 |

5x5

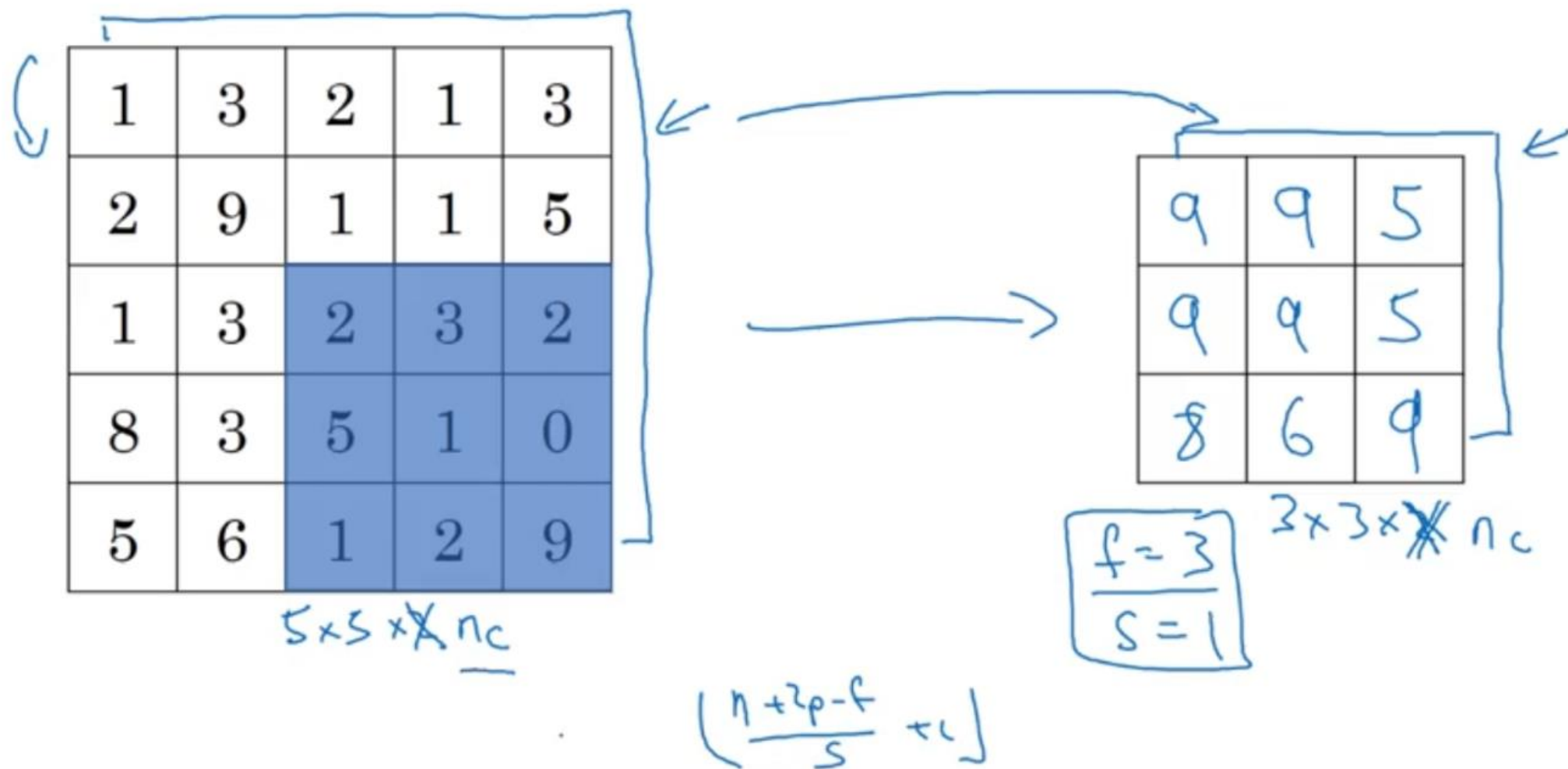
|   |   |   |
|---|---|---|
| 9 | 9 | 5 |
| 9 | 9 | 5 |
| 8 | 6 | 9 |

3x3

$$\frac{f=3}{s=1}$$

$$\left( \frac{n + 2p - f}{s} + 1 \right)$$

# Pooling layer: Max pooling



# Pooling layer: Average pooling

|   |   |   |   |
|---|---|---|---|
| 1 | 3 | 2 | 1 |
| 2 | 9 | 1 | 1 |
| 1 | 4 | 2 | 3 |
| 5 | 6 | 1 | 2 |



|      |      |
|------|------|
| 3.75 | 1.25 |
| 4    | 2    |

$f=2$   
s-

# Pooling layer: Average pooling

|   |   |   |   |
|---|---|---|---|
| 1 | 3 | 2 | 1 |
| 2 | 9 | 1 | 1 |
| 1 | 4 | 2 | 3 |
| 5 | 6 | 1 | 2 |



|      |      |
|------|------|
| 3.75 | 1.25 |
| 4    | 2    |

$$f=2$$

$$s=2$$

$$\underline{7 \times 7 \times 1000} \rightarrow 1 \times 1 \times 1000$$

# Summary of pooling

Hyperparameters:

$f$  : filter size

$$f=2, s=2$$

$s$  : stride

$$f=3, s=2$$

Max or average pooling

~~$\Rightarrow p$ : padding.~~

$$\begin{array}{c} n_H \times n_W \times \underline{n_C} \\ \downarrow \\ \left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \\ \times \underline{n_C} \end{array}$$

# Summary of pooling

Hyperparameters:

f : filter size

$$f=2, s=2$$

s : stride

$$f=3, s=2$$

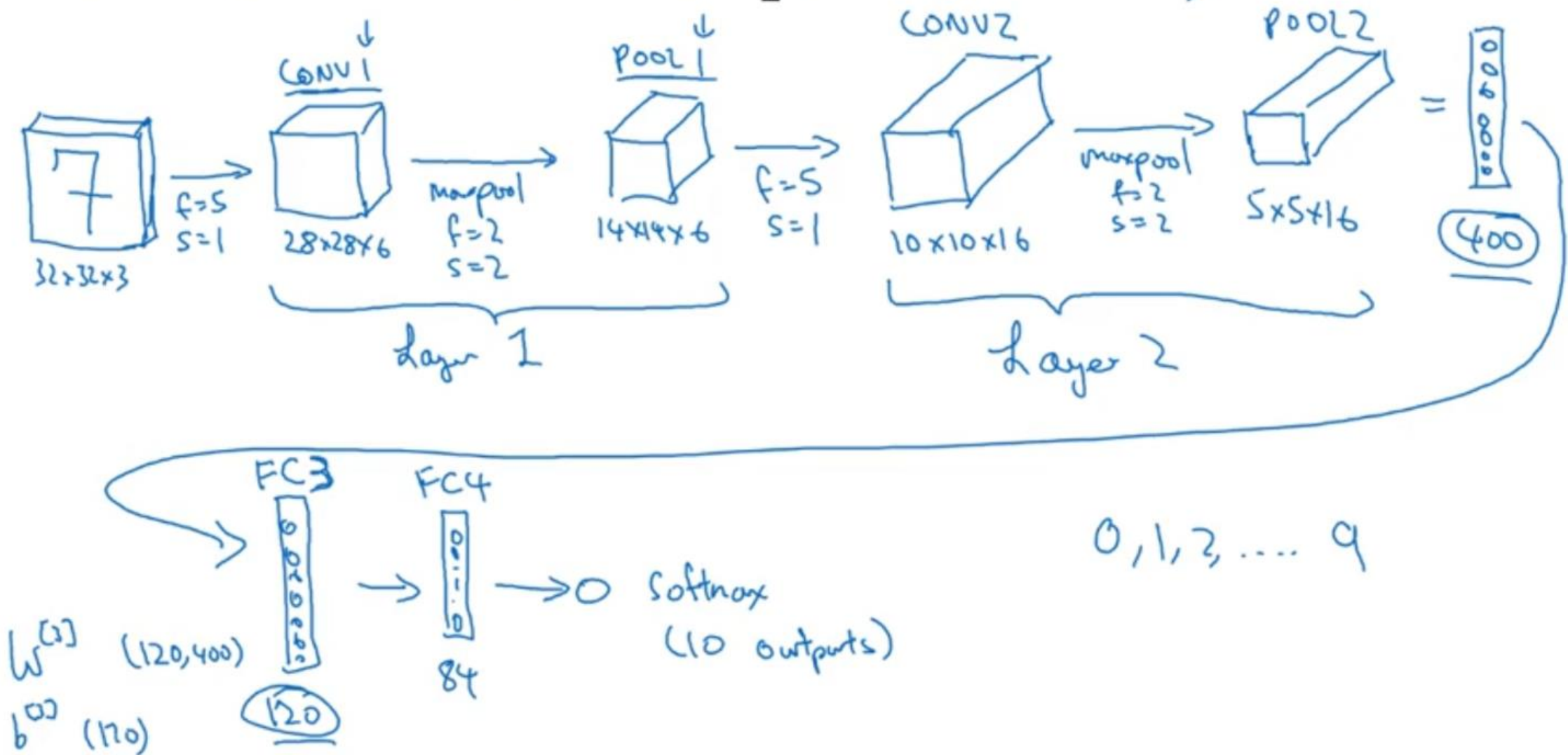
Max or average pooling

~~⇒ p: padding.~~

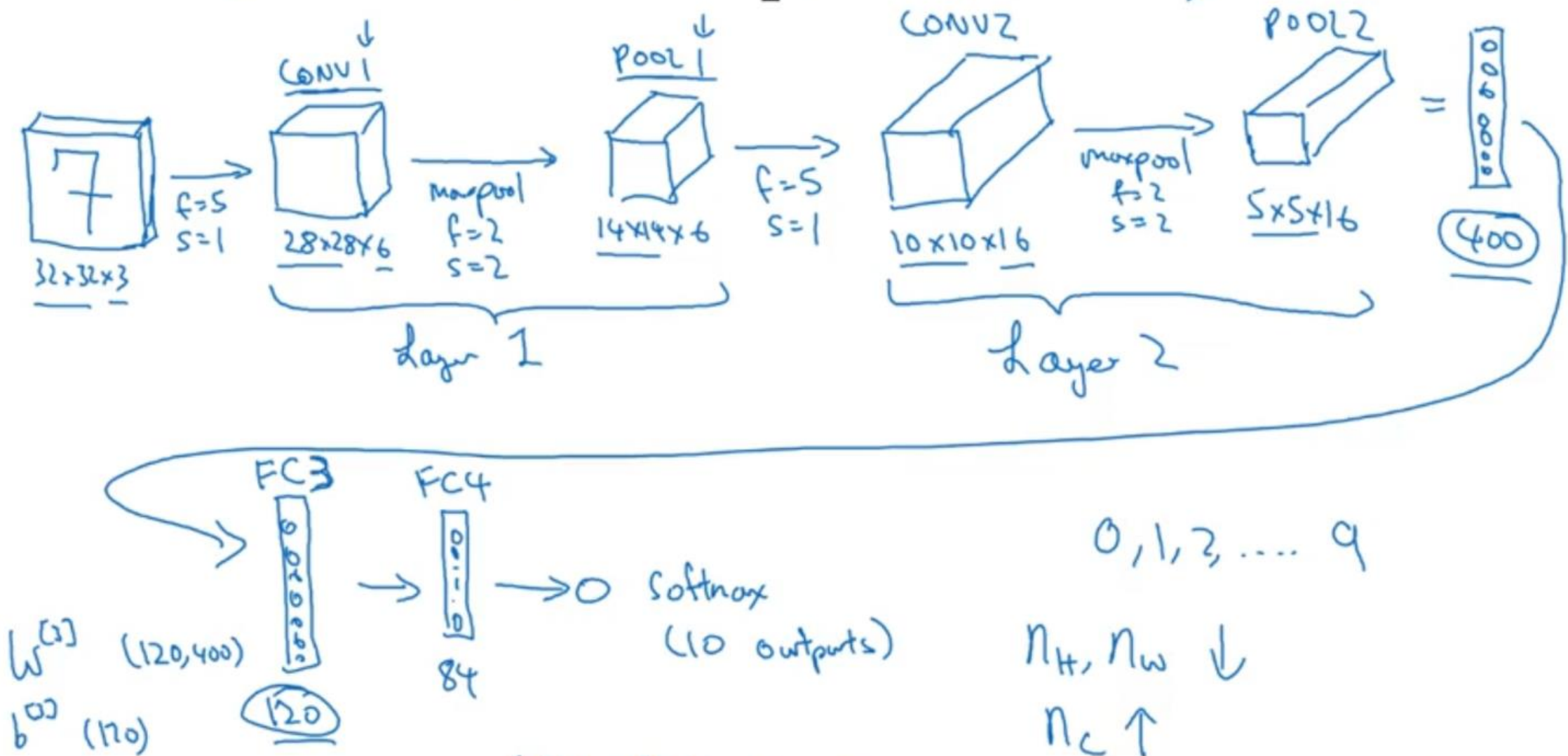
No parameters to learn!

$$\begin{array}{c} n_H \times n_W \times \underline{n_C} \\ \downarrow \\ \left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \\ \times \underline{n_C} \end{array}$$

# Neural network example (LeNet-5)



# Neural network example (LeNet-5)



CONV - POOL - CONV - POOL - FC - FC - FC - SOFTMAX

this neural network some more details  
of what are the activation shape,

Andrew Ng

# Neural network example

|                  | Activation shape | Activation Size   | # parameters  |
|------------------|------------------|-------------------|---------------|
| Input:           | (32,32,3)        | — 3,072 $a^{col}$ | 0             |
| CONV1 (f=5, s=1) | (28,28,8)        | <u>6,272</u>      | 208 ←         |
| POOL1            | (14,14,8)        | <u>1,568</u>      | 0 ←           |
| CONV2 (f=5, s=1) | (10,10,16)       | <u>1,600</u>      | 416 ←         |
| POOL2            | (5,5,16)         | <u>400</u>        | 0 ←           |
| FC3              | (120,1)          | <u>120</u>        | 48,001 }<br>} |
| FC4              | (84,1)           | <u>84</u>         | 10,081 }<br>} |
| Softmax          | (10,1)           | <u>10</u>         | 841           |



Starting from 9:45, please note that the calculation of the number of parameters is incorrect.

Here is a screenshot of that slide:

## Neural network example

|                  | Activation shape | Activation Size | # parameters |
|------------------|------------------|-----------------|--------------|
| Input:           | (32,32,3)        | 3,072 $a^{[0]}$ | 0            |
| CONV1 (f=5, s=1) | (28,28,8)        | 6,272           | 208 ←        |
| POOL1            | (14,14,8)        | 1,568           | 0 ←          |
| CONV2 (f=5, s=1) | (10,10,16)       | 1,600           | 416 ←        |
| POOL2            | (5,5,16)         | 400             | 0 ←          |
| FC3              | (120,1)          | 120             | 48,001 } ←   |
| FC4              | (84,1)           | 84              | 10,081 } ←   |
| Softmax          | (10,1)           | 10              | 841          |

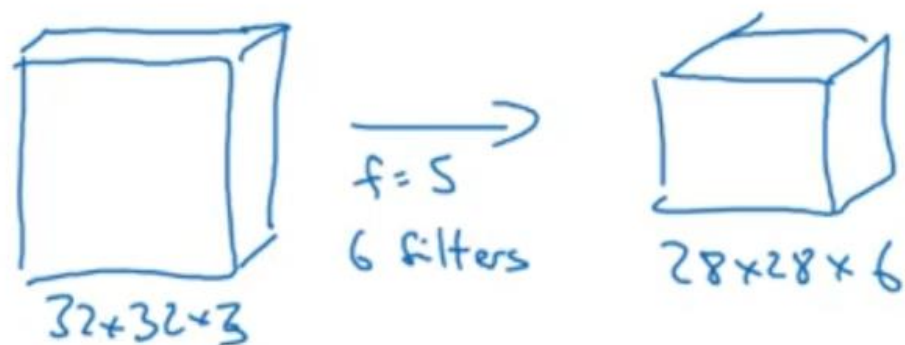
Andrew Ng

Here are the 5 typos:

1. 208 should be  $(5*5*3 + 1) * 8 = 608$
  2. 416 should be  $(5*5*8 + 1) * 16 = 3216$
  3. In the FC3, 48001 should be  $400*120 + 120 = 48120$ , since the bias should have 120 parameters, not 1
  4. Similarly, in the FC4, 10081 should be  $120*84 + 84$  (not 1) = 10164
- (Here, the bias is for the fully connected layer. In fully connected layers, there will be one bias for each neuron, so the bias become In FC3 there were 120 neurons so 120 biases.)
5. Finally, in the softmax, 841 should be  $84*10 + 10 = 850$

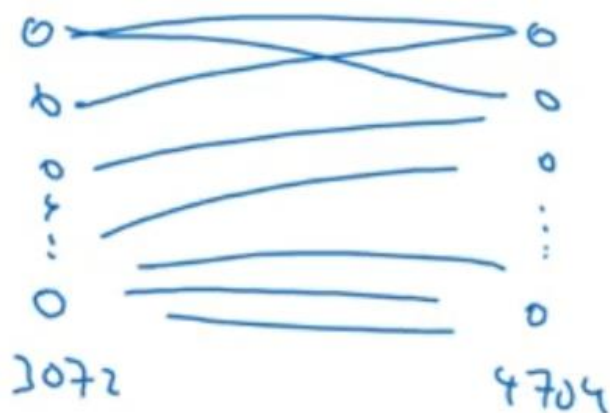
NEXT ITEM

# Why convolutions



3,072

4,704

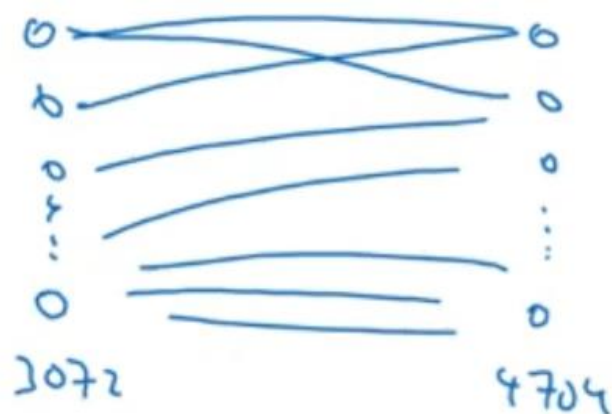
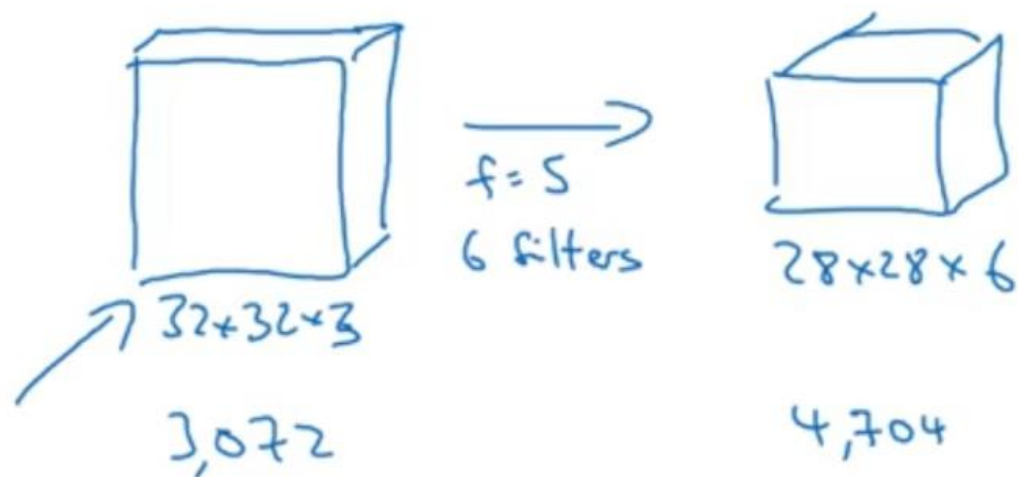


$$3,072 \times 4,704 \approx \underline{14M}$$

Andrew Ng

So, that's just a lot of parameters to train.

# Why convolutions



$$5 \times 5 = 25$$

$$26$$

$$6 \times 26 = 156 \text{ parameters}$$

$$3,072 \times 4,704 \approx \underline{14M}$$

# Why convolutions

|    |    |    |   |   |   |
|----|----|----|---|---|---|
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |

 $*$ 

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

*3x3*

 $=$ 

|   |    |    |   |
|---|----|----|---|
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |

**Parameter sharing:** A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

# Why convolutions

|    |    |    |   |   |   |
|----|----|----|---|---|---|
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |
| 10 | 10 | 10 | 0 | 0 | 0 |

\*

|   |   |    |
|---|---|----|
| 1 | 0 | -1 |
| 1 | 0 | -1 |
| 1 | 0 | -1 |

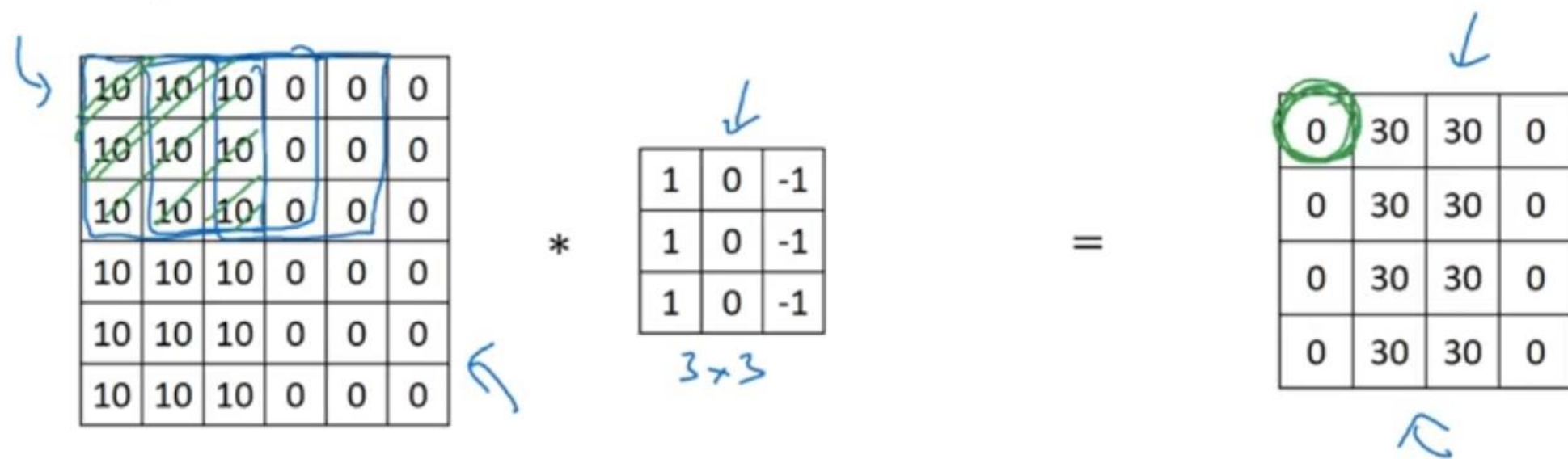
3x3

=

|   |    |    |   |
|---|----|----|---|
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |
| 0 | 30 | 30 | 0 |

**Parameter sharing:** A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

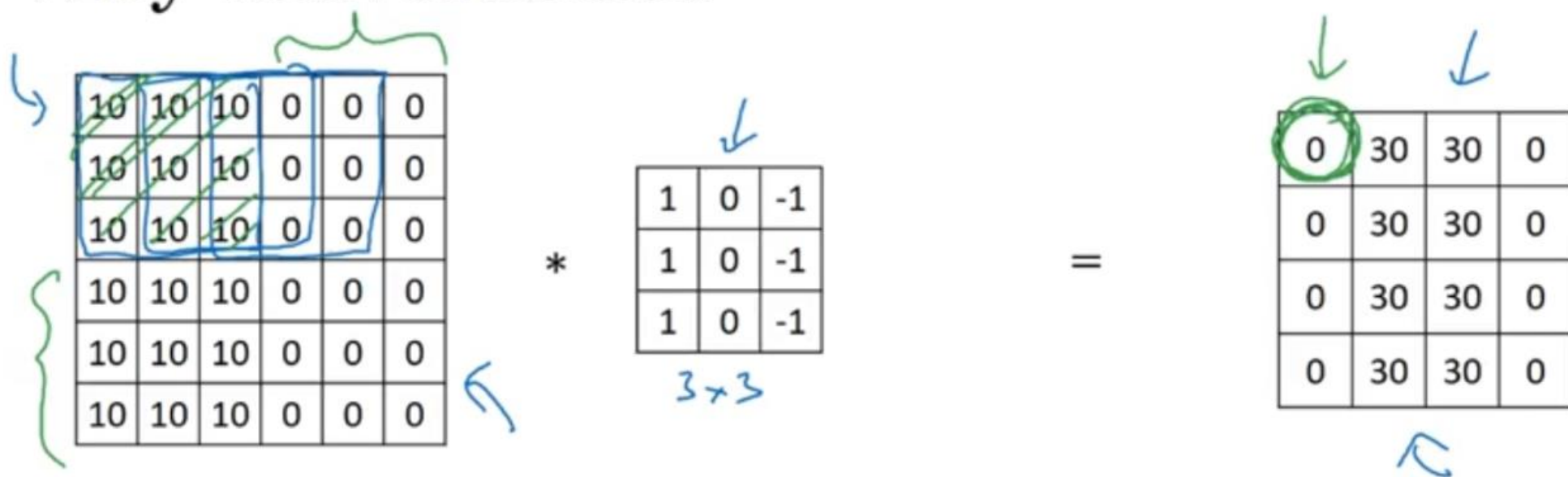
# Why convolutions



**Parameter sharing:** A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

→ **Sparsity of connections:** In each layer, each output value depends only on a small number of inputs.

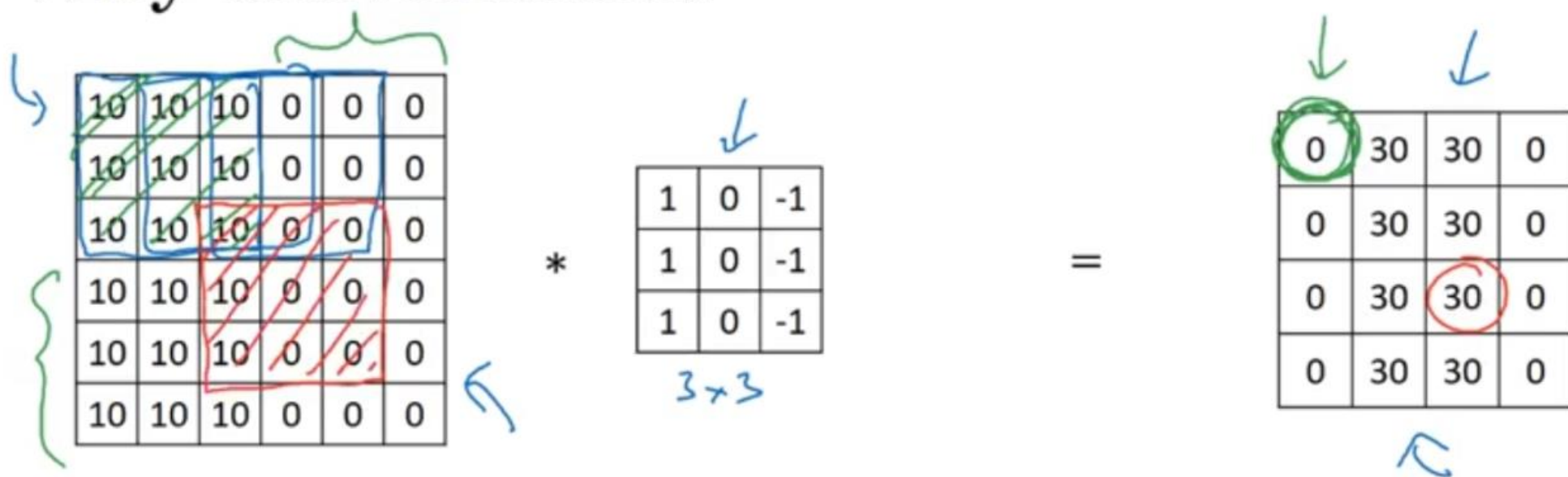
# Why convolutions



**Parameter sharing:** A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

→ **Sparsity of connections:** In each layer, each output value depends only on a small number of inputs.

# Why convolutions

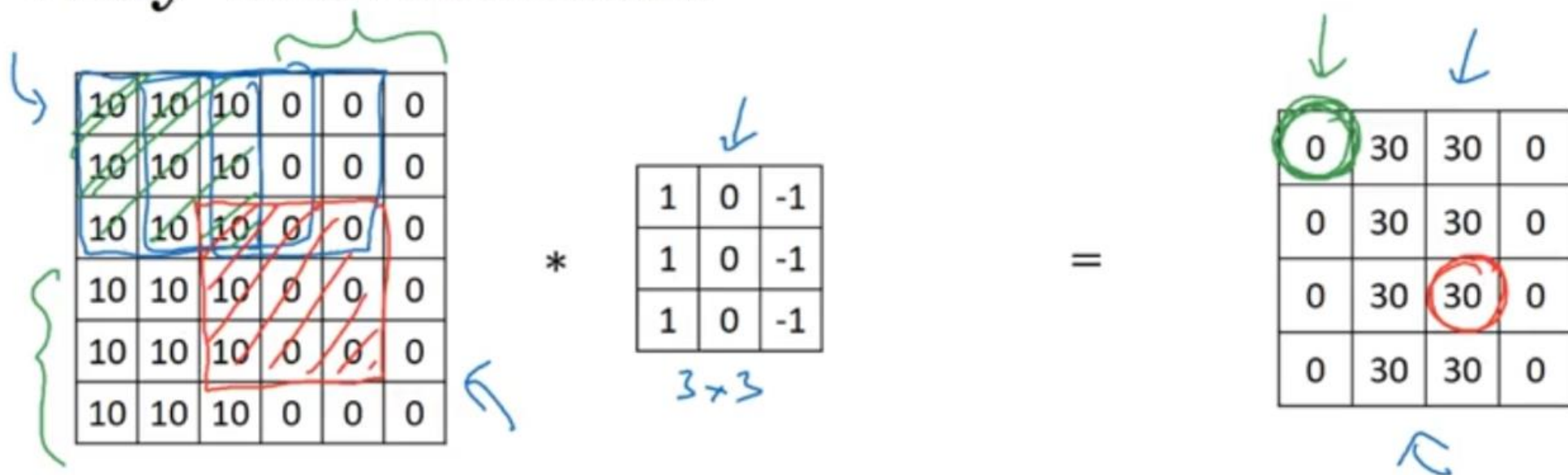


**Parameter sharing:** A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

→ **Sparsity of connections:** In each layer, each output value depends only on a small number of inputs.

# Why convolutions

trans



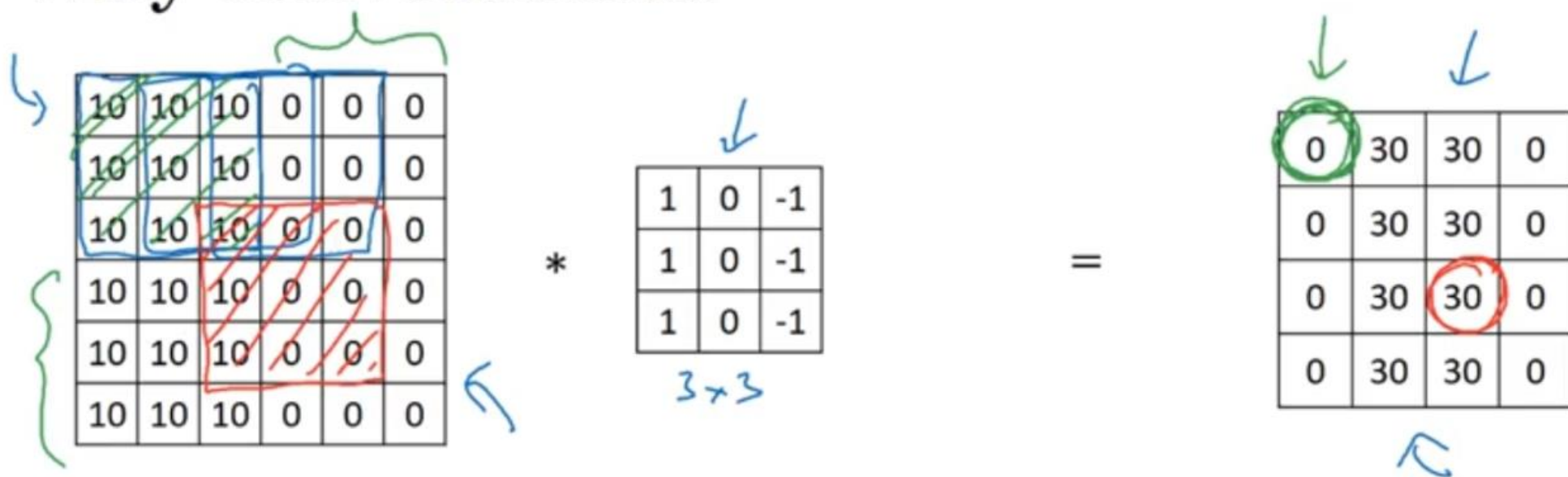
**Parameter sharing:** A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

→ **Sparsity of connections:** In each layer, each output value depends only on a small number of inputs.

Andrew Ng

# Why convolutions

translation invariance



**Parameter sharing:** A feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image.

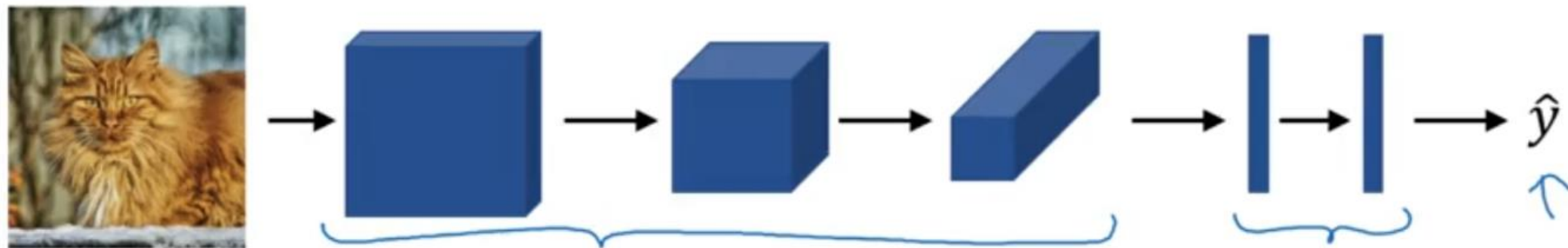
→ **Sparsity of connections:** In each layer, each output value depends only on a small number of inputs.

Andrew Ng

shifted a few pixels should result in pretty similar features and

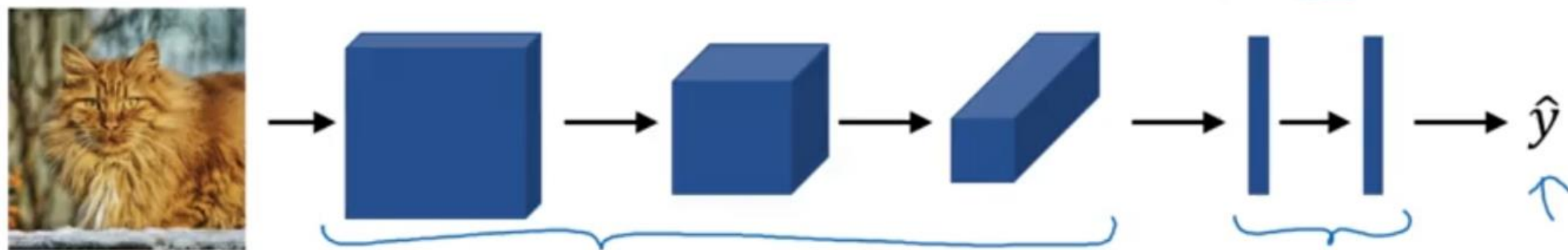
# Putting it together

Training set  $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$ .



# Putting it together

Training set  $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$ .



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Use gradient descent to optimize parameters to reduce  $J$